

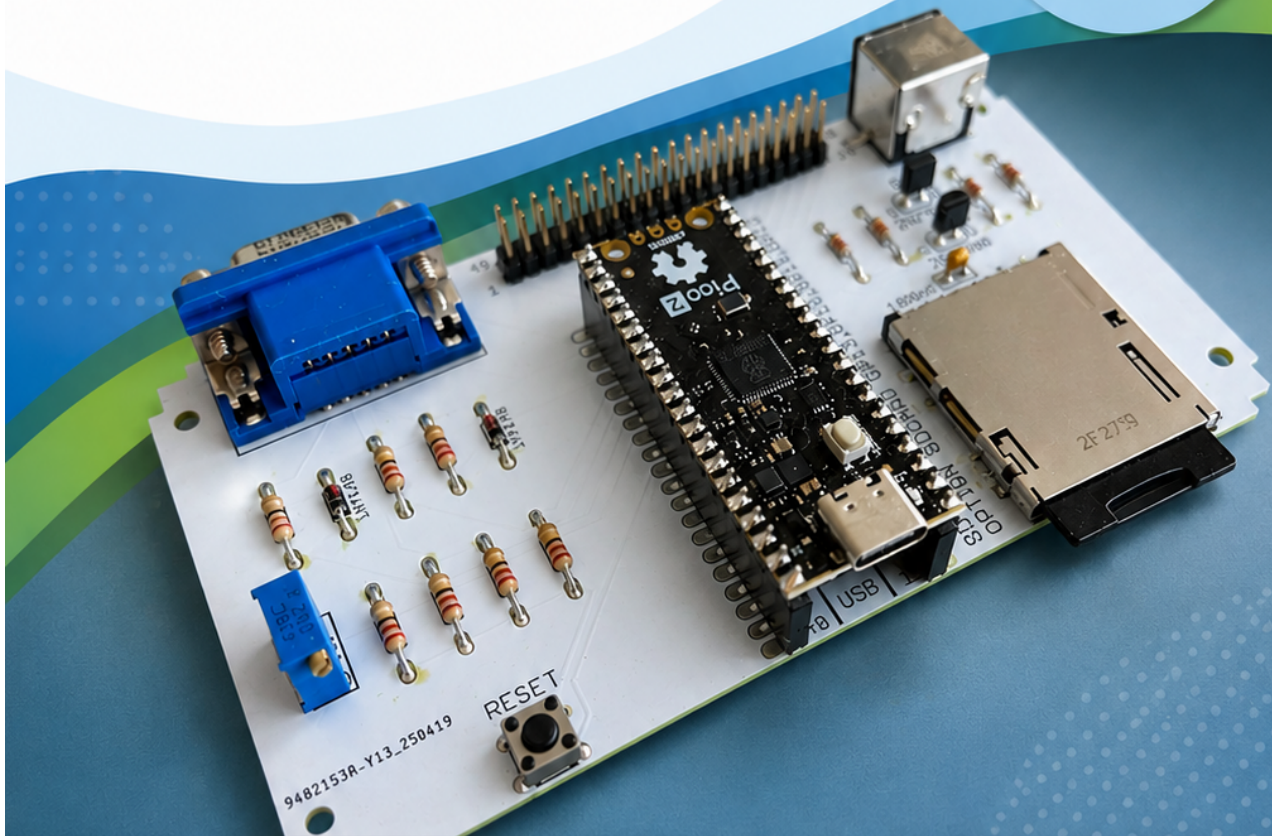
Mein erstes MMBasic Programm

Schritt für Schritt in die Welt
von MMBasic auf dem PicoMite
und Colour Maximite

>_

Programmieren
leicht gemacht!

Für Einsteiger
ohne Vorkenntnisse
geeignet!



Erste Schritte
mit MMBasic



Programme
eingeben & starten



Textausgabe
und Befehle



Beispiele & Übungen
zum Mitmachen

Mein erstes MMBasic Programm

Schritt für Schritt in die Welt von MMBasic

für PicoMite und Colour Maximate

Manfred Becker

Version 0.07

2026-07-12

Impressum

Autor: Manfred Becker

Projektseite:

<https://github.com/ManiBecker/MeinErstesMMBasicProgramm>

Dieses Tutorial entsteht Schritt für Schritt öffentlich auf GitHub.

MMBasic wurde von Geoff Graham entwickelt und von der MMBasic-Community weitergeführt und erweitert.

Table of Contents

Mein erstes MMBasic Programm	2
Impressum	3
Willkommen!	5
1. Hallo Welt!	6
2. Mit MMBasic rechnen	8
3. Variablen	13
4. Benutzereingaben mit INPUT	17
5. Entscheidungen mit IF	22
6. Schleifen mit FOR ... NEXT	28
7. Zufallszahlen	35
8. Unser erstes Spiel: Zahlenraten	38
9. Zahlenraten Version 2	42
10. Programme strukturieren mit SUB und FUNCTION	47
11. Arrays	54
12. Mit Texten arbeiten	61
13. Little Professor	70
14. Little Professor Version 2	76
15. Little Professor Version 3	84
16. Dateien speichern und laden	91
Grafik und Benutzeroberflächen	98
17. Die verschiedenen Grafikmodi	99
18. Farben und Schriftarten	105
19. Die ersten Grafikbefehle	111
20. Rechtecke, Kreise und Texte	119
21. Treffe die Scheibe	129
22. Treffe die Scheibe – Version 2	135
23. Treffe die Scheibe – Version 3	142
24. Eine analoge Bahnhofsuhr	153

Willkommen!

Warum dieses Tutorial?

Willkommen bei **Mein erstes MMBasic Programm**.

Dieses Tutorial begleitet dich Schritt für Schritt bei deinen ersten Programmen in MMBasic.

Du benötigst keine Programmiererfahrung. Alles wird anhand kleiner Beispiele erklärt, die du sofort selbst ausprobieren kannst.

Chapter 1. Hallo Welt!

1.1. Das lernst du

In diesem Kapitel lernst du:

- wie ein MMBasic-Programm aussieht
- wie Programme gestartet werden
- wie Text auf dem Bildschirm ausgegeben wird

Am Ende des Kapitels hast du dein erstes eigenes MMBasic-Programm geschrieben und ausgeführt.

1.2. Unser erstes Programm

Gib folgendes Programm ein:

```
PRINT "Hallo Welt!"
```

Starte das Programm anschließend.

Auf dem Bildschirm erscheint:

```
Hallo Welt!
```

Herzlichen Glückwunsch!

Du hast gerade dein erstes MMBasic-Programm geschrieben.

1.3. Schritt für Schritt erklärt

Schauen wir uns das Programm genauer an:

```
PRINT "Hallo Welt!"
```

Die Anweisung **PRINT** gibt Text auf dem Bildschirm aus.

Alles, was zwischen den Anführungszeichen steht, wird unverändert angezeigt.

Du kannst jeden beliebigen Text ausgeben lassen.

Zum Beispiel:

```
PRINT "Hallo MMBasic!"
```

oder

```
PRINT "Ich programmiere einen PicoMite."
```

1.4. Experimentiere!

Ändere den Text zwischen den Anführungszeichen.

Probiere verschiedene Meldungen aus.

Beispiele:

```
PRINT "Hallo Colour Maximate!"
```

```
PRINT "Heute lerne ich MMBasic."
```

Was passiert, wenn du den Text änderst?

1.5. Probier's selbst!

Versuche folgende Aufgaben:

1. Gib deinen Namen aus.
2. Gib deinen Wohnort aus.
3. Gib eine Begrüßung für einen Freund aus.
4. Gib drei verschiedene Meldungen nacheinander aus.

Beispiel:

```
PRINT "Hallo Welt!"  
PRINT "Willkommen bei MMBasic!"  
PRINT "Viel Spaß beim Programmieren!"
```

1.6. Zusammenfassung

In diesem Kapitel hast du die Anweisung **PRINT** kennengelernt.

Mit **PRINT** können Texte auf dem Bildschirm ausgegeben werden.

Im nächsten Kapitel lernen wir den integrierten Editor kennen und schreiben unsere ersten mehrzeiligen Programme.

Chapter 2. Mit MMBasic rechnen

2.1. Das lernst du

In diesem Kapitel lernst du:

- wie MMBasic Berechnungen ausführt
- die vier Grundrechenarten
- die Verwendung von Klammern
- einige nützliche mathematische Funktionen

Am Ende des Kapitels kannst du MMBasic für einfache und komplexere Berechnungen einsetzen.

2.2. MMBasic kann rechnen

Im letzten Kapitel haben wir mit **PRINT** Text auf dem Bildschirm ausgegeben.

Mit **PRINT** können aber nicht nur Texte, sondern auch Berechnungsergebnisse ausgegeben werden.

Gib einmal folgende Zeile ein:

```
PRINT 1+2
```

MMBasic antwortet:

```
3
```

Die Berechnung wird sofort ausgeführt.

2.3. Die vier Grundrechenarten

Addition:

```
PRINT 5+3
```

Ergebnis:

```
8
```

Subtraktion:

PRINT 10-4

Ergebnis:

6

Multiplikation:

PRINT 6*7

Ergebnis:

42

Division:

PRINT 20/4

Ergebnis:

5

2.4. Mehrere Rechenarten kombinieren

Natürlich können auch mehrere Rechenarten miteinander kombiniert werden.

PRINT 2+3*4

Ergebnis:

14

Vielleicht hast du 20 erwartet?

MMBasic beachtet die bekannten Rechenregeln aus der Mathematik:

Multiplikation und Division werden vor Addition und Subtraktion ausgeführt.

2.5. Klammern verwenden

Mit Klammern kann die Reihenfolge verändert werden.

```
PRINT (2+3)*4
```

Ergebnis:

```
20
```

Klammern helfen dabei, Berechnungen übersichtlicher zu gestalten und Missverständnisse zu vermeiden.

2.6. Potenzen berechnen

Mit dem Zeichen [^] können Potenzen berechnet werden.

```
PRINT 2^8
```

Ergebnis:

```
256
```

Ein weiteres Beispiel:

```
PRINT 10^3
```

Ergebnis:

```
1000
```

2.7. Die Quadratwurzel

Die Funktion `SQR()` berechnet die Quadratwurzel einer Zahl.

```
PRINT SQR(25)
```

Ergebnis:

5

Weiteres Beispiel:

```
PRINT SQR(144)
```

Ergebnis:

12

2.8. Der Betrag einer Zahl

Die Funktion **ABS()** liefert den Betrag einer Zahl.

```
PRINT ABS(-10)
```

Ergebnis:

10

```
PRINT ABS(10)
```

Ergebnis:

10

2.9. Experimentiere!

Probiere einige eigene Berechnungen aus:

```
PRINT 123+456  
PRINT 50*20  
PRINT (10+20)*3  
PRINT 3^4  
PRINT SQR(81)
```

Was passiert?

2.10. Probier's selbst!

Versuche folgende Aufgaben:

1. Berechne $15 + 27$.
2. Berechne $100 - 37$.
3. Berechne $12 * 12$.
4. Berechne die Quadratwurzel von 225.
5. Berechne $(5 + 7) * 8$.

2.11. Zusammenfassung

In diesem Kapitel hast du gelernt, wie MMBasic Berechnungen ausführt.

Du kennst nun:

- die vier Grundrechenarten
- Klammern
- Potenzen
- die Funktionen `SQR()` und `ABS()`

Im nächsten Kapitel lernen wir Variablen kennen. Damit können wir Werte speichern und später wiederverwenden.

Chapter 3. Variablen

3.1. Das lernst du

In diesem Kapitel lernst du:

- was Variablen sind
- wie Werte gespeichert werden
- wie Variablen in Berechnungen verwendet werden
- wie Variablen ihren Inhalt ändern können

Am Ende des Kapitels kannst du eigene Werte speichern und weiterverarbeiten.

3.2. Was ist eine Variable?

Eine Variable ist ein Speicherplatz für einen Wert.

Du kannst dir eine Variable wie eine beschriftete Schachtel vorstellen.

In diese Schachtel kannst du einen Wert hineinlegen und später wieder darauf zugreifen.

3.3. Unsere erste Variable

Gib folgende Zeilen ein:

```
A=10  
PRINT A
```

Ausgabe:

```
10
```

MMBasic speichert den Wert 10 in der Variablen A.

Mit PRINT kann der gespeicherte Wert angezeigt werden.

3.4. Mehrere Variablen verwenden

Natürlich können mehrere Variablen gleichzeitig verwendet werden.

```
A=10  
B=20  
  
PRINT A
```

```
PRINT B
```

Ausgabe:

```
10  
20
```

3.5. Mit Variablen rechnen

Variablen können in Berechnungen verwendet werden.

```
A=10  
B=20  
  
PRINT A+B
```

Ausgabe:

```
30
```

Weitere Beispiele:

```
PRINT A-B  
PRINT A*B  
PRINT B/A
```

3.6. Variablen verändern

Der Inhalt einer Variablen kann jederzeit geändert werden.

```
A=10  
PRINT A  
  
A=25  
PRINT A
```

Ausgabe:

```
10  
25
```

Der alte Wert wird dabei überschrieben.

3.7. Der Inhalt einer Variablen kann berechnet werden

Besonders interessant wird es, wenn eine Variable mit ihrem eigenen Inhalt verändert wird.

```
A=10  
A=A+1  
  
PRINT A
```

Ausgabe:

```
11
```

Schauen wir uns die zweite Zeile genauer an:

```
A=A+1
```

Zuerst berechnet MMBasic die rechte Seite:

```
10+1
```

Danach wird das Ergebnis wieder in A gespeichert.

3.8. Zähler erstellen

Auf diese Weise kann ein einfacher Zähler aufgebaut werden.

```
A=0  
  
A=A+1  
PRINT A  
  
A=A+1  
PRINT A  
  
A=A+1  
PRINT A
```

Ausgabe:

```
1  
2  
3
```

3.9. Experimentiere!

Probiere folgende Beispiele aus:

```
A=100  
B=50  
  
PRINT A+B  
PRINT A-B  
PRINT A*B
```

Was passiert?

3.10. Probier's selbst!

Versuche folgende Aufgaben:

1. Speichere dein Alter in einer Variablen.
2. Speichere dein Geburtsjahr in einer Variablen.
3. Addiere zwei Zahlen mit Hilfe von Variablen.
4. Erhöhe eine Variable dreimal um 1.

3.11. Zusammenfassung

In diesem Kapitel hast du Variablen kennengelernt.

Du kannst nun:

- Werte speichern
- Werte verändern
- mit Variablen rechnen

Im nächsten Kapitel lernen wir Benutzereingaben mit INPUT kennen.

Chapter 4. Benutzereingaben mit INPUT

4.1. Das lernst du

In diesem Kapitel lernst du:

- wie Daten vom Benutzer eingegeben werden
- wie die Anweisung INPUT verwendet wird
- wie Eingaben in Variablen gespeichert werden
- wie Eingaben weiterverarbeitet werden

Am Ende des Kapitels kannst du eigene interaktive Programme schreiben.

4.2. Warum Benutzereingaben wichtig sind

Bisher standen alle Werte fest im Programm.

Zum Beispiel:

```
A=10  
B=20  
PRINT A+B
```

Das Ergebnis ist immer gleich.

Interessanter wird es, wenn der Benutzer die Werte selbst eingeben kann.

4.3. Unsere erste Eingabe

Die Anweisung **INPUT** wartet auf eine Eingabe des Benutzers.

Gib folgendes ein:

```
INPUT A  
PRINT A
```

MMBasic zeigt ein Fragezeichen an:

```
?
```

Gib nun beispielsweise folgende Zahl ein:

42

Danach erscheint:

42

4.4. Eingaben in Variablen speichern

Die eingegebene Zahl wird in der Variablen gespeichert.

```
INPUT A  
PRINT A
```

Wenn du 100 eingibst:

100

wird dieser Wert in der Variablen **A** gespeichert.

4.5. Mit Eingaben rechnen

Eingegebene Werte können sofort weiterverarbeitet werden.

```
INPUT A  
PRINT A*2
```

Beispiel:

? 21
42

4.6. Zwei Werte eingeben

Natürlich können auch mehrere Werte eingegeben werden.

```
INPUT A  
INPUT B  
  
PRINT A+B
```

Beispiel:

```
? 10  
? 20  
30
```

4.7. Eine persönliche Begrüßung

MMBasic kann auch Texte verarbeiten.

Dazu verwenden wir eine String-Variable.

String-Variablen erkennt man am Dollarzeichen am Ende des Namens.

```
INPUT N$  
PRINT "Hallo ";N$
```

Beispiel:

```
? Manfred  
Hallo Manfred
```

MMBasic erlaubt aussagekräftige Variablennamen.

Statt kurzer Namen wie

```
A  
B  
X
```

können auch längere Namen verwendet werden:



```
ALTER  
NAME$  
WOHNORT$  
PUNKTE
```

Ein Variablenname darf mit einem Buchstaben beginnen und kann anschließend weitere Buchstaben, Ziffern und den Unterstrich `_` enthalten.

Für Textvariablen wird häufig ein Dollarzeichen `$` am Ende des Namens verwendet:

```
NAME$
```

```
WOHNORT$  
TEXT$
```

4.8. Texte und Variablen kombinieren

Mit einem Semikolon können Texte und Variablen gemeinsam ausgegeben werden.

```
INPUT NAME$  
PRINT "Willkommen ";NAME$  
PRINT "Schön, dass du da bist."
```

4.9. Unser erstes interaktives Programm

```
PRINT "Wie heißt du?"  
INPUT NAME$  
  
PRINT "Hallo ";NAME$  
PRINT "Willkommen bei MMBasic!"
```

Beispiel:

```
Wie heißt du?  
? Manfred  
  
Hallo Manfred  
Willkommen bei MMBasic!
```

4.10. Experimentiere!

Probiere folgende Änderungen aus:

- Gib dein Alter ein.
- Addiere zwei eingegebene Zahlen.
- Gib deinen Wohnort ein.
- Erweitere die Begrüßung um weitere Texte.

4.11. Probier's selbst!

Versuche folgende Aufgaben:

1. Frage den Benutzer nach seinem Namen.
2. Frage den Benutzer nach seinem Alter.

3. Addiere zwei eingegebene Zahlen.
4. Gib eine persönliche Begrüßung aus.

4.12. Zusammenfassung

In diesem Kapitel hast du die Anweisung **INPUT** kennengelernt.

Du kannst nun:

- Zahlen eingeben
- Texte eingeben
- Eingaben speichern
- Eingaben weiterverarbeiten

Im nächsten Kapitel lernen wir Entscheidungen mit **IF ... THEN** kennen.

Chapter 5. Entscheidungen mit IF

5.1. Das lernst du

In diesem Kapitel lernst du:

- wie Programme Entscheidungen treffen
- wie die Anweisung IF verwendet wird
- wie Werte miteinander verglichen werden
- wie Programme unterschiedlich auf Eingaben reagieren

Am Ende des Kapitels kannst du Programme schreiben, die auf Benutzereingaben reagieren und unterschiedliche Aktionen ausführen.

5.2. Warum Entscheidungen wichtig sind

Bisher wurden unsere Programme immer von oben nach unten ausgeführt.

Ein Programm soll aber oft unterschiedlich reagieren.

Zum Beispiel:

- Ist ein Benutzer volljährig?
- Wurde die richtige Antwort eingegeben?
- Ist eine Zahl größer als 100?

Für solche Aufgaben verwendet man die Anweisung **IF**.

5.3. Unsere erste Entscheidung

Gib folgendes Programm ein:

```
A=10  
  
IF A=10 THEN  
    PRINT "Richtig!"  
ENDIF
```

Ausgabe:

```
Richtig!
```

Die Bedingung ist erfüllt.

Deshalb wird die Anweisung zwischen **THEN** und **ENDIF** ausgeführt.

5.4. Wenn die Bedingung nicht erfüllt ist

Ändere das Programm:

```
A=5  
  
IF A=10 THEN  
    PRINT "Richtig!"  
ENDIF
```

Ausgabe:

Es erscheint keine Ausgabe.

Die Bedingung ist nicht erfüllt.

Deshalb wird der Bereich zwischen THEN und ENDIF übersprungen.

5.5. ELSE - Was passiert sonst?

Oft soll ein Programm nicht nur auf richtige Eingaben reagieren.

Es soll auch etwas tun, wenn die Bedingung nicht erfüllt ist.

Dafür gibt es ELSE.

```
A=5  
  
IF A=10 THEN  
    PRINT "Richtig!"  
ELSE  
    PRINT "Falsch!"  
ENDIF
```

Ausgabe:

Falsch!

Ändere nun den Wert von A auf 10.

Welche Ausgabe erscheint jetzt?

5.6. Werte vergleichen

Mit IF können verschiedene Vergleiche durchgeführt werden.

Gleich:

```
IF A=10 THEN  
PRINT "Gleich"  
ENDIF
```

Ungleich:

```
IF A<>10 THEN  
PRINT "Ungleich"  
ENDIF
```

Größer:

```
IF A>10 THEN  
PRINT "Groesser"  
ENDIF
```

Kleiner:

```
IF A<10 THEN  
PRINT "Kleiner"  
ENDIF
```

Größer oder gleich:

```
IF A>=10 THEN  
PRINT "Groesser oder gleich"  
ENDIF
```

Kleiner oder gleich:

```
IF A<=10 THEN  
PRINT "Kleiner oder gleich"  
ENDIF
```

5.7. Alter prüfen

Mit Benutzereingaben werden Entscheidungen besonders interessant.


```
PRINT "Wie alt bist du?"
INPUT ALTER

IF ALTER>=18 THEN
PRINT "Du bist volljaehrig."
ELSE
PRINT "Du bist minderjaehrig."
ENDIF
```

Beispiel:

```
Wie alt bist du?
? 25

Du bist volljaehrig.
```

5.8. Texte vergleichen

Auch Texte können verglichen werden.

```
PRINT "Wie heisst du?"
INPUT NAME$

IF NAME$="MANFRED" THEN
PRINT "Hallo Manfred!"
ELSE
PRINT "Hallo unbekannter Besucher!"
ENDIF
```

5.9. Unser erstes Quiz

```
PRINT "Wieviel ist 2+3?"
INPUT ANTWORT

IF ANTWORT=5 THEN
PRINT "Richtig!"
ELSE
PRINT "Leider falsch."
ENDIF
```

Beispiel:

```
Wieviel ist 2+3?
? 5
```

Richtig!

5.10. Mehrere Entscheidungen

Ein Programm kann beliebig viele Entscheidungen enthalten.

```
INPUT NOTE

IF NOTE=1 THEN
PRINT "Sehr gut"
ENDIF

IF NOTE=2 THEN
PRINT "Gut"
ENDIF

IF NOTE=3 THEN
PRINT "Befriedigend"
ENDIF
```

5.11. ELSEIF verwenden

Oft müssen mehrere Möglichkeiten geprüft werden.

Dafür eignet sich **ELSEIF**.

```
INPUT NOTE

IF NOTE=1 THEN
    PRINT "Sehr gut"
ELSEIF NOTE=2 THEN
    PRINT "Gut"
ELSEIF NOTE=3 THEN
    PRINT "Befriedigend"
ELSE
    PRINT "Unbekannte Note"
ENDIF
```

Beispiel:

? 2

Gut

5.12. Experimentiere!

Probiere folgende Beispiele aus:

```
INPUT ZAHL
```

```
IF ZAHL>100 THEN
```

```
PRINT "Die Zahl ist groesser als 100."
```

```
ELSE
```

```
PRINT "Die Zahl ist kleiner oder gleich 100."
```

```
ENDIF
```

Was passiert bei verschiedenen Eingaben?

5.13. Probier's selbst!

Versuche folgende Aufgaben:

1. Frage das Alter ab und prüfe, ob die Person volljährig ist.
2. Frage einen Namen ab und gib eine persönliche Begrüßung aus.
3. Prüfe, ob eine Zahl größer als 100 ist.
4. Erstelle ein kleines Rechenquiz.
5. Erweitere das Quiz um eine Ausgabe für falsche Antworten.

5.14. Zusammenfassung

In diesem Kapitel hast du die Anweisung IF kennengelernt.

Du kannst nun:

Werte vergleichen Entscheidungen treffen auf Benutzereingaben reagieren unterschiedliche Programmabläufe mit ELSE erstellen

Im nächsten Kapitel lernen wir Schleifen mit FOR ... NEXT kennen.

Chapter 6. Schleifen mit FOR ... NEXT

6.1. Das lernst du

In diesem Kapitel lernst du:

- wie Schleifen funktionieren
- wie die Anweisung FOR verwendet wird
- wie Programme Anweisungen mehrfach ausführen können
- wie Zählvariablen verwendet werden

Am Ende des Kapitels kannst du Programme schreiben, die Aufgaben automatisch wiederholen.

6.2. Warum Schleifen wichtig sind

Stell dir vor, du möchtest zehnmal "Hallo Welt!" ausgeben.

Ohne Schleife müsstest du schreiben:

```
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"  
PRINT "Hallo Welt!"
```

Das funktioniert, ist aber sehr umständlich.

Mit einer Schleife geht es deutlich einfacher.

6.3. Unsere erste Schleife

```
FOR I=1 TO 10  
  PRINT "Hallo Welt!"  
NEXT I
```

Ausgabe:

```
Hallo Welt!  
Hallo Welt!
```

```
Hallo Welt!  
Hallo Welt!  
Hallo Welt!  
Hallo Welt!  
Hallo Welt!  
Hallo Welt!  
Hallo Welt!  
Hallo Welt!
```

6.4. Was passiert hier?

Schauen wir uns die Schleife genauer an:

```
FOR I=1 TO 10
```

MMBasic erzeugt eine Zählvariable mit dem Namen **I**.

Der Wert beginnt bei 1.

Nach jedem Durchlauf wird der Wert automatisch erhöht.

Sobald der Wert 10 erreicht hat, endet die Schleife.

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

6.5. Die Zählvariable ausgeben

Die aktuelle Nummer kann jederzeit ausgegeben werden.

```
FOR I=1 TO 10  
PRINT I  
NEXT I
```

Ausgabe:

```
1
```

```
2  
3  
4  
5  
6  
7  
8  
9  
10
```

6.6. Mit der Zählvariable rechnen

Die Zählvariable kann auch in Berechnungen verwendet werden.

```
FOR I=1 TO 10  
PRINT I*I  
NEXT I
```

Ausgabe:

```
1  
4  
9  
16  
25  
36  
49  
64  
81  
100
```

6.7. Das kleine Einmaleins

Mit einer Schleife lässt sich schnell ein kleines Einmaleins erzeugen.

```
FOR I=1 TO 10  
PRINT I;" x 5 = ";I*5  
NEXT I
```

Ausgabe:

```
1 x 5 = 5  
2 x 5 = 10  
3 x 5 = 15  
...
```

$$10 \times 5 = 50$$

6.8. Rückwärts zählen

Schleifen können auch rückwärts laufen.

Dazu wird STEP verwendet.

```
FOR I=10 TO 1 STEP -1  
PRINT I  
NEXT I
```

Ausgabe:

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1
```

6.9. Größere Schritte

Mit STEP können auch andere Schrittweiten verwendet werden.

```
FOR I=0 TO 20 STEP 2  
PRINT I  
NEXT I
```

Ausgabe:

```
0  
2  
4  
6  
8  
10  
12  
14  
16  
18
```

6.10. Unser erster Countdown

```
FOR I=10 TO 1 STEP -1  
PRINT I  
NEXT I  
  
PRINT "Start!"
```

Ausgabe:

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
Start!
```

6.11. Schleifen und Entscheidungen

Schleifen und IF lassen sich hervorragend kombinieren.

```
FOR I=1 TO 10  
  
IF I MOD 2 = 0 THEN  
PRINT I  
ENDIF  
  
NEXT I
```

Ausgabe:

```
2  
4  
6  
8  
10
```


6.12. Eine mathematische Funktion berechnen

Schleifen werden häufig verwendet, um mathematische Funktionen für viele Werte zu berechnen.

Betrachten wir die Funktion:

$$y = 0.5 \cdot x^2 + x + 0.5$$

Wir möchten die Funktion für alle Werte von 0 bis 10 berechnen.

```
FOR X=0 TO 10  
  
    Y=0.5*X^2+X+0.5  
  
    PRINT "X=";X;"  Y=";Y  
  
NEXT X
```

Ausgabe:

X=0	Y=0.5
X=1	Y=2
X=2	Y=4.5
X=3	Y=8
X=4	Y=12.5
X=5	Y=18
X=6	Y=24.5
X=7	Y=32
X=8	Y=40.5
X=9	Y=50
X=10	Y=60.5

Mit einer Schleife können auf diese Weise auch Tabellen für komplexere Funktionen erzeugt werden.

Frühe Heimcomputer und Taschenrechner wurden häufig genau für solche Berechnungen eingesetzt.



Mit einer Schleife lassen sich auch Tabellen für mathematische Funktionen erzeugen.

Solche Wertetabellen wurden früher häufig verwendet, um Funktionen zu untersuchen oder später als Graph darzustellen.

6.13. Experimentiere!

Probiere folgende Änderungen aus:

Gib die Zahlen von 1 bis 20 aus. Zähle von 100 bis 0 herunter. Gib alle Vielfachen von 3 aus. Berechne die Quadratzahlen von 1 bis 20.

6.14. Probier's selbst!

Versuche folgende Aufgaben:

1. Gib die Zahlen von 1 bis 50 aus.
2. Gib nur die geraden Zahlen von 1 bis 20 aus.
3. Erstelle einen Countdown von 20 bis 1.
4. Erzeuge das Einmaleins der Zahl 7.
5. Berechne die Quadratzahlen von 1 bis 15.

6.15. Zusammenfassung

In diesem Kapitel hast du die FOR-Schleife kennengelernt.

Du kannst nun:

Anweisungen mehrfach ausführen
vorwärts zählen rückwärts zählen
Schrittweiten mit STEP verwenden
Schleifen mit IF kombinieren

Im nächsten Kapitel schreiben wir unser erstes größeres Programm.

Chapter 7. Zufallszahlen

7.1. Das lernst du

In diesem Kapitel lernst du:

- wie Zufallszahlen erzeugt werden
- wie die Funktion RND verwendet wird
- wie Zufallszahlen auf einen gewünschten Bereich angepasst werden

Am Ende des Kapitels kannst du Würfel, Münzwürfe und kleine Spiele programmieren.

7.2. Warum Zufallszahlen wichtig sind

Viele Programme benötigen Zufallszahlen.

Zum Beispiel:

- Spiele
- Quizprogramme
- Simulationen
- Würfel
- Lottozahlen

MMBasic stellt dafür die Funktion **RND** bereit.

7.3. Eine Zufallszahl erzeugen

Gib folgendes Programm ein:

```
PRINT RND
```

Bei jeder Ausführung erscheint eine andere Zahl.

Zum Beispiel:

```
0.38142
```

oder

```
0.92471
```

7.4. Zufallszahlen mehrfach erzeugen

Mit einer Schleife können mehrere Zufallszahlen ausgegeben werden.

```
FOR I=1 TO 10
  PRINT RND
NEXT I
```

7.5. Einen Würfel simulieren

Ein Würfel besitzt die Werte 1 bis 6.

```
PRINT INT(RND*6)+1
```

Mögliche Ausgabe:

```
4
```

7.6. Mehrfach würfeln

```
FOR I=1 TO 10
  PRINT INT(RND*6)+1
NEXT I
```

7.7. Münze werfen

```
IF INT(RND*2)=0 THEN
  PRINT "Kopf"
ELSE
  PRINT "Zahl"
ENDIF
```

7.8. Lottozahlen erzeugen

```
FOR I=1 TO 6
  PRINT INT(RND*49)+1
NEXT I
```

Hinweis:

Die Zahlen können mehrfach auftreten. Eine vollständige Lotto-Simulation werden wir später erstellen.

7.9. Ein Ratespiel

```
GEHEIM=INT(RND*10)+1  
  
PRINT "Rate eine Zahl von 1 bis 10"  
  
INPUT TIPP  
  
IF TIPP=GEHEIM THEN  
    PRINT "Richtig!"  
ELSE  
    PRINT "Leider falsch."  
ENDIF
```

7.10. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Zufallszahlen zu erzeugen
- Würfel zu simulieren
- Münzen zu werfen
- kleine Spiele zu programmieren

Im nächsten Kapitel schreiben wir unser erstes größeres Programm.

Chapter 8. Unser erstes Spiel: Zahlenraten

8.1. Das lernst du

In diesem Kapitel lernst du:

- wie mehrere Befehle zu einem vollständigen Programm kombiniert werden
- wie Zufallszahlen verwendet werden
- wie Benutzereingaben verarbeitet werden
- wie Programme auf richtige und falsche Antworten reagieren

Am Ende des Kapitels hast du dein erstes kleines Spiel programmiert.

8.2. Die Spielidee

Der Computer denkt sich eine Zahl aus.

Der Spieler versucht diese Zahl zu erraten.

Danach verrät das Programm, ob der Tipp richtig war.

8.3. Das komplette Programm

```
GEHEIM=INT(RND*10)+1

PRINT "Ich habe mir eine Zahl von 1 bis 10 ausgedacht."

INPUT TIPP

IF TIPP=GEHEIM THEN
  PRINT "Richtig!"
ELSE
  PRINT "Leider falsch."
ENDIF
```

8.4. Das Programm verstehen

Die erste Zeile erzeugt eine Zufallszahl:

```
GEHEIM=INT(RND*10)+1
```

Die Funktion **RND** liefert eine Zufallszahl.

Durch Multiplikation mit 10 und anschließendes Abrunden mit **INT()** entsteht eine ganze Zahl

zwischen 0 und 9.

Durch Addition von 1 wird daraus eine Zahl zwischen 1 und 10.

Danach fordert das Programm den Benutzer zur Eingabe eines Tipps auf:

```
INPUT TIPP
```

Anschließend werden Tipp und Zufallszahl verglichen:

```
IF TIPP=GEHEIM THEN  
    ...  
ENDIF
```

Stimmen beide Werte überein, gewinnt der Spieler.

8.5. Das Spiel testen

Starte das Programm mehrmals.

Du wirst feststellen, dass sich die geheime Zahl bei jedem Programmlauf ändert.

Das liegt daran, dass jedes Mal eine neue Zufallszahl erzeugt wird.

8.6. Mehrere Spielrunden

Mit einer Schleife können mehrere Spielrunden hintereinander gespielt werden.

```
FOR RUNDE=1 TO 5  
  
    GEHEIM=INT(RND*10)+1  
  
    PRINT  
    PRINT "Runde ";RUNDE  
  
    INPUT TIPP  
  
    IF TIPP=GEHEIM THEN  
        PRINT "Richtig!"  
    ELSE  
        PRINT "Leider falsch."  
    ENDIF  
  
NEXT RUNDE
```

Das Spiel wird nun fünfmal hintereinander ausgeführt.

8.7. Punkte sammeln

Mit einer zusätzlichen Variablen können wir die Anzahl der Treffer zählen.

```
PUNKTE=0

FOR RUNDE=1 TO 5

    GEHEIM=INT(RND*10)+1

    PRINT
    PRINT "Runde ";RUNDE

    INPUT TIPP

    IF TIPP=GEHEIM THEN
        PRINT "Richtig!"
        PUNKTE=PUNKTE+1
    ELSE
        PRINT "Leider falsch."
    ENDIF

NEXT RUNDE

PRINT
PRINT "Du hast ";PUNKTE;" Punkte erreicht."
```

8.8. Das Spiel verbessern

Das Spiel funktioniert bereits erstaunlich gut.

Allerdings hat der Spieler pro Runde nur einen einzigen Versuch.

Außerdem erfährt er nicht, ob sein Tipp zu groß oder zu klein war.

Diese Verbesserungen werden wir in einem späteren Kapitel umsetzen.

8.9. Ideen für Erweiterungen

Vielleicht möchtest du das Spiel bereits jetzt erweitern.

Zum Beispiel:

- Zahlen von 1 bis 20 verwenden
- Zahlen von 1 bis 100 verwenden
- zehn Spielrunden durchführen
- zwei Punkte für einen Treffer vergeben

- den Namen des Spielers abfragen

8.10. Experimentiere!

Probiere folgende Änderungen aus:

- Erhöhe den Zahlenbereich auf 20.
- Erhöhe den Zahlenbereich auf 100.
- Spiele zehn Runden statt fünf.
- Vergib zwei Punkte für jeden Treffer.

Welche Auswirkungen haben die Änderungen?

8.11. Probier's selbst!

Versuche folgende Aufgaben:

1. Erweitere das Spiel auf zehn Runden.
2. Zähle die Treffer.
3. Gib die Gesamtpunktzahl aus.
4. Verwende Zahlen von 1 bis 50.
5. Frage den Namen des Spielers ab und begrüße ihn persönlich.

8.12. Zusammenfassung

In diesem Kapitel hast du dein erstes vollständiges Spiel programmiert.

Dabei wurden viele der bisher gelernten Befehle verwendet:

- PRINT
- INPUT
- Variablen
- IF
- FOR
- RND

Du hast gesehen, wie aus einzelnen Befehlen ein vollständiges Programm entsteht.

Im nächsten Kapitel werden wir das Spiel weiter verbessern und dem Spieler Hinweise geben, ob sein Tipp zu groß oder zu klein war.

Chapter 9. Zahlenraten Version 2

9.1. Das lernst du

In diesem Kapitel lernst du:

- wie ein Spiel mehrere Versuche erlauben kann
- wie Hinweise ausgegeben werden
- wie die Anzahl der Versuche gezählt wird
- wie die Schleife DO ... LOOP UNTIL verwendet wird

Am Ende des Kapitels hast du ein deutlich verbessertes Zahlenratespiel programmiert.

9.2. Das Problem

In unserem ersten Zahlenratespiel hatte der Spieler nur einen einzigen Versuch.

War die Eingabe falsch, war das Spiel sofort beendet.

Das ist nicht besonders spannend.

Wir möchten dem Spieler deshalb mehrere Versuche erlauben.

9.3. Die neue Spielidee

Der Computer denkt sich wieder eine Zahl aus.

Der Spieler darf so lange raten, bis er die richtige Zahl gefunden hat.

Zusätzlich erhält er hilfreiche Hinweise:

- Zu klein!
- Zu groß!
- Richtig!

9.4. Eine neue Schleife

Für solche Aufgaben eignet sich die Schleife DO ... LOOP.

```
DO  
  
    PRINT "Hallo Welt!"  
  
LOOP
```

Vorsicht!

Dieses Programm läuft endlos und kann nur mit CTRL+C beendet werden.

9.5. Die Schleife beenden

Eine Schleife kann mit einer Bedingung beendet werden.

```
DO  
  
    INPUT A  
  
LOOP UNTIL A=10
```

Das Programm fordert solange eine Zahl an, bis die Zahl 10 eingegeben wird.

9.6. Das neue Zahlenratespiel

```
GEHEIM=INT(RND*100)+1  
  
PRINT "Ich habe mir eine Zahl von 1 bis 100 ausgedacht."  
  
DO  
  
    INPUT TIPP  
  
    IF TIPP<GEHEIM THEN  
        PRINT "Zu klein!"  
    ELSEIF TIPP>GEHEIM THEN  
        PRINT "Zu gross!"  
    ELSE  
        PRINT "Richtig!"  
    ENDIF  
  
LOOP UNTIL TIPP=GEHEIM
```

9.7. Das Programm verstehen

Wie bisher wird zunächst eine Zufallszahl erzeugt.

```
GEHEIM=INT(RND*100)+1
```

Danach beginnt die Schleife:

```
DO
```

```
...  
LOOP UNTIL TIPP=GEHEIM
```

Die Schleife wird solange wiederholt, bis der Spieler die richtige Zahl eingegeben hat.

9.8. Die Anzahl der Versuche zählen

Wie viele Versuche benötigt der Spieler?

Das können wir mit einer Variablen zählen.

```
VERSUCHE=0  
  
DO  
  
    VERSUCHE=VERSUCHE+1  
  
    INPUT TIPP  
  
    IF TIPP<GEHEIM THEN  
        PRINT "Zu klein!"  
    ELSEIF TIPP>GEHEIM THEN  
        PRINT "Zu gross!"  
    ELSE  
        PRINT "Richtig!"  
    ENDIF  
  
LOOP UNTIL TIPP=GEHEIM  
  
PRINT  
PRINT "Du hast ";VERSUCHE;" Versuche benoetigt."
```

9.9. Das Spiel ausprobieren

Starte das Programm mehrfach.

Wie viele Versuche benötigst du?

Schaffst du es in weniger als fünf Versuchen?

9.10. Eine persönliche Begrüßung

Das Spiel kann den Spieler auch persönlich begrüßen.

```
PRINT "Wie heisst du?"  
INPUT NAME$
```

```

PRINT
PRINT "Hallo ";NAME$

GEHEIM=INT(RND*100)+1
VERSUCHE=0

DO

    VERSUCHE=VERSUCHE+1

    INPUT TIPP

    IF TIPP<GEHEIM THEN
        PRINT "Zu klein!"
    ELSEIF TIPP>GEHEIM THEN
        PRINT "Zu gross!"
    ELSE
        PRINT "Richtig!"
    ENDIF

LOOP UNTIL TIPP=GEHEIM

PRINT
PRINT "Du hast ";VERSUCHE;" Versuche benoetigt."

```



Viele Programme entstehen nicht auf einmal.

Oft beginnt man mit einer einfachen Version und erweitert diese Schritt für Schritt.

Genau so ist auch unser Zahlenratespiel entstanden.

9.11. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende Zahlen von 1 bis 50.
- Verwende Zahlen von 1 bis 1000.
- Begrenze die Anzahl der Versuche.
- Vergib Punkte für besonders gute Ergebnisse.

9.12. Probier's selbst!

Versuche folgende Aufgaben:

1. Begrenze die Anzahl der Versuche auf 10.
2. Gib nach dem Spiel die geheime Zahl aus.

3. Vergib Punkte abhängig von der Anzahl der Versuche.
4. Frage nach jedem Spiel, ob noch einmal gespielt werden soll.
5. Erweitere das Spiel um einen Highscore.

9.13. Zusammenfassung

In diesem Kapitel hast du unser erstes Spiel verbessert.

Du hast gelernt:

- die Schleife DO ... LOOP UNTIL zu verwenden
- Hinweise auszugeben
- Versuche zu zählen
- Programme schrittweise zu erweitern

Im nächsten Kapitel lernen wir Unterprogramme kennen. Damit können größere Programme übersichtlicher aufgebaut werden.

Chapter 10. Programme strukturieren mit SUB und FUNCTION

10.1. Das lernst du

In diesem Kapitel lernst du:

- Programme in kleinere Bausteine aufzuteilen
- eigene Unterprogramme mit SUB zu erstellen
- eigene Funktionen mit FUNCTION zu schreiben
- Parameter an Unterprogramme und Funktionen zu übergeben

Am Ende des Kapitels kannst du eigene Befehle und Funktionen entwickeln.

10.2. Warum Programme strukturieren?

Unsere Programme werden langsam größer.

Das Zahlenratespiel aus dem letzten Kapitel besteht bereits aus vielen Zeilen Code.

Bei größeren Programmen wird es zunehmend schwieriger, den Überblick zu behalten.

Deshalb teilen Programmierer ihre Programme in kleinere, überschaubare Bausteine auf.

MMBasic stellt dafür SUB und FUNCTION zur Verfügung.

10.3. Unsere erste SUB

Eine SUB fasst mehrere Anweisungen zu einem Unterprogramm zusammen.

```
SUB Begrueessung  
    PRINT "Hallo Welt!"  
END SUB  
Begrueessung
```

Ausgabe:

```
Hallo Welt!
```

Die Anweisungen zwischen SUB und END SUB werden erst ausgeführt, wenn die SUB aufgerufen wird.

10.4. Eine SUB mehrfach aufrufen

Eine SUB kann beliebig oft verwendet werden.

```
SUB Begrueessung
```

```
    PRINT "Hallo Welt!"
```

```
END SUB
```

```
Begrueessung
```

```
Begrueessung
```

```
Begrueessung
```

Ausgabe:

```
Hallo Welt!
```

```
Hallo Welt!
```

```
Hallo Welt!
```

10.5. Eine SUB mit Parametern

Oft soll ein Unterprogramm unterschiedliche Werte verarbeiten.

Dazu können Parameter verwendet werden.

```
SUB Begrueessung(NAME$)
```

```
    PRINT "Hallo ";NAME$
```

```
END SUB
```

```
Begrueessung("Manfred")
```

```
Begrueessung("Geoff")
```

```
Begrueessung("Peter")
```

Ausgabe:

```
Hallo Manfred
```

```
Hallo Geoff
```

```
Hallo Peter
```


10.6. Warum FUNCTION?

Eine SUB führt Anweisungen aus.

Manchmal möchten wir jedoch einen Wert berechnen und zurückgeben.

Dafür gibt es FUNCTION.

10.7. Unsere erste FUNCTION

```
FUNCTION Quadrat(X)  
  
    Quadrat=X*X  
  
END FUNCTION  
  
PRINT Quadrat(5)
```

Ausgabe:

25

Die Funktion erhält einen Wert als Parameter und liefert das Quadrat dieses Wertes zurück.

10.8. Weitere Funktionen

```
FUNCTION Verdoppeln(X)  
  
    Verdoppeln=X*2  
  
END FUNCTION  
  
PRINT Verdoppeln(10)
```

Ausgabe:

20

10.9. Funktionen in Berechnungen verwenden

Der Rückgabewert einer Funktion kann direkt in Berechnungen verwendet werden.

```
FUNCTION Quadrat(X)
```

```
    Quadrat=X*X  
  
END FUNCTION  
  
PRINT Quadrat(5)+Quadrat(3)
```

Ausgabe:

34

10.10. Eine Würfelfunktion

Mit Funktionen lassen sich eigene BASIC-Befehle erstellen.

```
FUNCTION Wuerfel()  
  
    Wuerfel=INT(RND*6)+1  
  
END FUNCTION  
  
PRINT Wuerfel()  
PRINT Wuerfel()  
PRINT Wuerfel()
```

Mögliche Ausgabe:

4
1
6

Bei jedem Aufruf wird eine neue Zufallszahl erzeugt.

10.11. Eine mathematische Funktion

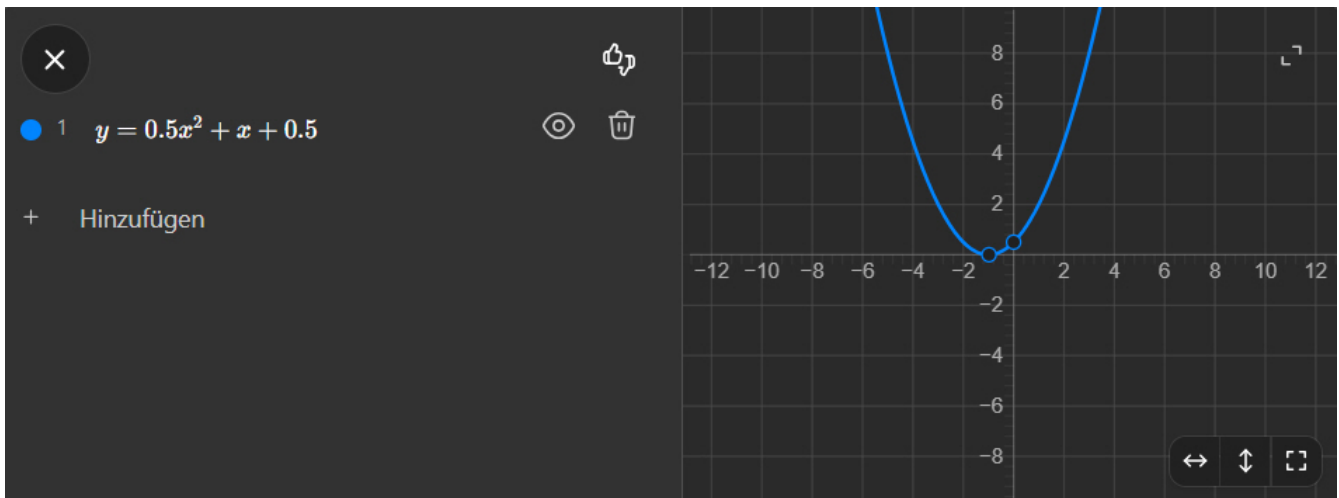
Eigene Funktionen eignen sich hervorragend für mathematische Berechnungen.

```
FUNCTION F(X)  
  
    F=0.5*X^2+X+0.5  
  
END FUNCTION  
  
FOR X=0 TO 10
```

```
PRINT X;" ";F(X)
```

```
NEXT X
```

Die Funktion berechnet dabei: $y = 0.5x^2 + x + 0.5$



10.12. Das Zahlenratespiel verbessern

Im letzten Kapitel wurde die geheime Zahl direkt erzeugt:

```
GEHEIM=INT(RND*100)+1
```

Mit einer Funktion wird der Programmcode besser lesbar.

```
FUNCTION NeueZahl()  
  
    NeueZahl=INT(RND*100)+1  
  
END FUNCTION  
  
GEHEIM=NeueZahl()
```

Der Programmierer erkennt sofort, was die Zeile bewirkt.

10.13. Wann verwendet man SUB?

Eine SUB eignet sich für Programmteile, die Aufgaben ausführen.

Beispiele:

- Begrüßungen ausgeben
- Menüs anzeigen
- Bildschirmausgaben erzeugen

- Daten einlesen

10.14. Wann verwendet man FUNCTION?

Eine FUNCTION eignet sich für Programmteile, die einen Wert berechnen.

Beispiele:

- Flächen berechnen
- Temperaturen umrechnen
- Zufallszahlen erzeugen
- mathematische Funktionen berechnen

10.15. Experimentiere!

Probiere folgende Änderungen aus:

- Schreibe eine Funktion zur Berechnung des Dreifachen einer Zahl.
- Schreibe eine Funktion für die Berechnung eines Kreises.
- Erweitere die Würfelfunktion.
- Erstelle eine SUB für eine persönliche Begrüßung.

10.16. Probier's selbst!

Versuche folgende Aufgaben:

1. Schreibe eine Funktion zur Berechnung des Würfels einer Zahl.
2. Schreibe eine Funktion zur Berechnung des Umfangs eines Quadrats.
3. Erstelle eine SUB, die fünfmal "MMBasic" ausgibt.
4. Verwende eine Funktion innerhalb einer FOR-Schleife.
5. Baue die Funktion NeueZahl() in dein Zahlenratespiel ein.

10.17. Zusammenfassung

In diesem Kapitel hast du gelernt, Programme zu strukturieren.

Du kennst nun:

- SUB ... END SUB
- FUNCTION ... END FUNCTION
- Parameter
- Rückgabewerte

Mit diesen Werkzeugen lassen sich auch größere Programme übersichtlich aufbauen.

Im nächsten Kapitel werden wir unser Wissen nutzen, um ein weiteres Spiel oder ein größeres Praxisprojekt zu entwickeln.

Chapter 11. Arrays

11.1. Das lernst du

In diesem Kapitel lernst du:

- was ein Array ist
- wie Arrays mit DIM angelegt werden
- wie Werte in Arrays gespeichert werden
- wie Arrays mit FOR-Schleifen verarbeitet werden
- wie Zahlen- und Textarrays verwendet werden

Am Ende des Kapitels kannst du viele Werte übersichtlich verwalten.

11.2. Das Problem

In unserem Zahlenratespiel könnten mehrere Spieler gegeneinander antreten.

Für jeden Spieler möchten wir die Anzahl der benötigten Versuche speichern.

Ohne Arrays könnte das so aussehen:

```
VERSUCH0=5  
VERSUCH1=8  
VERSUCH2=3  
VERSUCH3=6  
VERSUCH4=4
```

Das funktioniert.

Aber was passiert bei 20 Spielern?

Oder 100 Spielern?

Es muss einen besseren Weg geben.

11.3. Die Lösung: Arrays

Ein Array ist eine Sammlung von Variablen mit demselben Namen.

Jedes Element besitzt eine Nummer, den sogenannten Index.

Bevor ein Array verwendet werden kann, muss es mit DIM angelegt werden.

```
DIM VERSUCH(4)
```

MMBasic nummeriert Array-Elemente standardmäßig ab 0.

Das Array

```
DIM VERSUCH(4)
```



enthält die Elemente

```
VERSUCH(0)  
VERSUCH(1)  
VERSUCH(2)  
VERSUCH(3)  
VERSUCH(4)
```

also insgesamt fünf Speicherplätze.

11.4. Werte speichern

Werte werden wie bei normalen Variablen gespeichert.

```
DIM VERSUCH(4)
```

```
VERSUCH(0)=5  
VERSUCH(1)=8  
VERSUCH(2)=3  
VERSUCH(3)=6  
VERSUCH(4)=4
```

11.5. Werte ausgeben

Auf einzelne Elemente kann direkt zugegriffen werden.

```
PRINT VERSUCH(0)  
PRINT VERSUCH(1)  
PRINT VERSUCH(2)
```

Ausgabe:

```
5  
8  
3
```

11.6. Arrays und FOR-Schleifen

Arrays und FOR-Schleifen sind ein perfektes Team.

```
DIM VERSUCH(4)

FOR I=0 TO 4

    INPUT VERSUCH(I)

NEXT I
```

Die eingegebenen Werte werden automatisch in den Array-Elementen gespeichert.

11.7. Alle Werte ausgeben

```
FOR I=0 TO 4

    PRINT VERSUCH(I)

NEXT I
```

11.8. Den Durchschnitt berechnen

Arrays eignen sich hervorragend für Berechnungen.

```
DIM VERSUCH(4)

SUMME=0

FOR I=0 TO 4

    INPUT VERSUCH(I)

    SUMME=SUMME+VERSUCH(I)

NEXT I

PRINT
PRINT "Durchschnitt:"
PRINT SUMME/5
```

11.9. Den besten Spieler finden

Nun suchen wir die kleinste Anzahl von Versuchen.


```

DIM VERSUCH(4)

FOR I=0 TO 4

    INPUT VERSUCH(I)

NEXT I

BEST=VERSUCH(0)

FOR I=1 TO 4

    IF VERSUCH(I)<BEST THEN
        BEST=VERSUCH(I)
    ENDIF

NEXT I

PRINT
PRINT "Bester Spieler:"
PRINT BEST;" Versuche"

```

11.10. Arrays initialisieren

Arrays können bereits beim Anlegen mit Werten gefüllt werden.

```
DIM NAME$(2)=("Manfred","Geoff","Peter")
```

Anschließend können die Werte direkt verwendet werden.

```

PRINT NAME$(0)
PRINT NAME$(1)
PRINT NAME$(2)

```

Ausgabe:

```

Manfred
Geoff
Peter

```

11.11. Textarrays

Natürlich können auch Texte gespeichert werden.

```
DIM SPIELER$(4)

SPIELER$(0)="Manfred"
SPIELER$(1)="Geoff"
SPIELER$(2)="Peter"
SPIELER$(3)="Tom"
SPIELER$(4)="Sarah"
```

Ausgabe:

```
FOR I=0 TO 4

  PRINT SPIELER$(I)

NEXT I
```

Ausgabe:

```
Manfred
Geoff
Peter
Tom
Sarah
```

11.12. Würfelergebnisse speichern

In Kapitel 7 haben wir einen Würfel simuliert.

Mit einem Array können wir mehrere Würfe speichern.

```
DIM WURF(9)

FOR I=0 TO 9

  WURF(I)=INT(RND*6)+1

NEXT I

FOR I=0 TO 9

  PRINT WURF(I)

NEXT I
```

Mögliche Ausgabe:

4
2
6
1
5
3
2
4
6
1

11.13. Eine Wertetabelle speichern

In Kapitel 6 haben wir eine mathematische Funktion berechnet.

Mit Arrays können wir die Ergebnisse speichern.

```
DIM Y(10)

FOR X=0 TO 10

    Y(X)=0.5*X^2+X+0.5

NEXT X

FOR X=0 TO 10

    PRINT X;" ";Y(X)

NEXT X
```

11.14. Mehrdimensionale Arrays

Ein Array kann mehrere Dimensionen besitzen.

Stell dir eine Tabelle mit Zeilen und Spalten vor.

Für eine kleine Notentabelle könnte man schreiben:

```
DIM NOTE(4,2)
```

Der erste Index beschreibt den Schüler.

Der zweite Index beschreibt das Fach.

NOTE(2,1)=1

Schüler Nummer 2 hat im Fach Nummer 1 die Note 1 erhalten.

11.15. Experimentiere!

Probiere folgende Änderungen aus:

- Speichere zehn Würfelergebnisse.
- Speichere die Punkte von zehn Spielern.
- Berechne einen Durchschnitt.
- Vergrößere ein Array.

11.16. Probier's selbst!

Versuche folgende Aufgaben:

1. Speichere die Alter von fünf Personen.
2. Gib alle Werte mit einer FOR-Schleife aus.
3. Berechne die Summe aller Werte.
4. Ermittle den größten Wert.
5. Speichere fünf Namen in einem Textarray.

11.17. Zusammenfassung

In diesem Kapitel hast du Arrays kennengelernt.

Du kannst nun:

- Arrays mit DIM anlegen
- Werte speichern
- Werte auslesen
- Arrays mit FOR-Schleifen verarbeiten
- Zahlen- und Textarrays verwenden
- mehrdimensionale Arrays anlegen

Arrays gehören zu den wichtigsten Werkzeugen der Programmierung und werden in vielen späteren Projekten verwendet.

Im nächsten Kapitel beschäftigen wir uns mit Zeichenketten und lernen, Texte zu bearbeiten und auszuwerten.

Chapter 12. Mit Texten arbeiten

12.1. Das lernst du

In diesem Kapitel lernst du:

- Texte in Variablen zu speichern
- Texte einzugeben und auszugeben
- die Länge eines Textes zu bestimmen
- Teile eines Textes auszulesen
- Texte in Groß- und Kleinbuchstaben umzuwandeln
- in Texten zu suchen
- Zahlen und Texte ineinander umzuwandeln
- Datum und Uhrzeit zu verwenden

Am Ende des Kapitels kannst du Texte in deinen Programmen verarbeiten und auswerten.

12.2. Zahlen und Texte

Bisher haben wir hauptsächlich mit Zahlen gearbeitet.

Zum Beispiel:

```
ALTER=60  
PUNKTE=125
```

Programme arbeiten aber nicht nur mit Zahlen.

Oft müssen auch Texte verarbeitet werden.

Zum Beispiel:

- Namen
- Adressen
- Dateinamen
- Antworten auf Fragen
- Meldungen

Texte werden in MMBasic in String-Variablen gespeichert.

String-Variablen erkennt man am Dollarzeichen am Ende des Namens.

```
NAME$="Manfred"
```

```
ORT$="Schwetzingen"
```

12.3. Wie lang darf ein Text sein?

Normale Zeichenketten können in MMBasic maximal 255 Zeichen enthalten.

Für die meisten Programme ist das vollkommen ausreichend.

Beispiele:

- Namen
- Dateinamen
- Benutzereingaben
- Meldungen
- kurze Texte

```
NAME$="Manfred"
```

Für sehr lange Texte stellt MMBasic sogenannte Long Strings zur Verfügung.

Diese werden wir später kennenlernen.

Für die meisten Programme reichen normale Zeichenketten jedoch völlig aus.

12.4. Texte ausgeben

Texte können wie Zahlen ausgegeben werden.

```
NAME$="Manfred"
```

```
PRINT NAME$
```

Ausgabe:

```
Manfred
```

12.5. Texte eingeben

Mit INPUT können auch Texte eingegeben werden.

```
INPUT NAME$
```

```
PRINT "Hallo ";NAME$
```

Beispiel:

```
? Manfred
```

```
Hallo Manfred
```

12.6. Texte verbinden

Mehrere Texte können in einer Ausgabe kombiniert werden.

```
VORNAME$="Manfred"
```

```
NACHNAME$="Becker"
```

```
PRINT VORNAME$;" ";NACHNAME$
```

Ausgabe:

```
Manfred Becker
```

12.7. Die Länge eines Textes bestimmen

Mit LEN() kann die Anzahl der Zeichen ermittelt werden.

```
NAME$="Manfred"
```

```
PRINT LEN(NAME$)
```

Ausgabe:

```
7
```

Jeder Buchstabe zählt als ein Zeichen.

Auch Leerzeichen werden mitgezählt.

12.8. Das erste Zeichen ermitteln

Mit LEFT\$() können Zeichen vom Anfang eines Textes gelesen werden.

```
NAME$="Manfred"
```

```
PRINT LEFT$(NAME$,1)
```

Ausgabe:

```
M
```

12.9. Mehrere Zeichen vom Anfang lesen

```
NAME$="Manfred"
```

```
PRINT LEFT$(NAME$,3)
```

Ausgabe:

```
Man
```

12.10. Zeichen vom Ende lesen

Mit RIGHT\$() können Zeichen vom Ende eines Textes gelesen werden.

```
NAME$="Manfred"
```

```
PRINT RIGHT$(NAME$,3)
```

Ausgabe:

```
red
```

12.11. Zeichen aus der Mitte lesen

Mit MID\$() können Zeichen aus der Mitte eines Textes gelesen werden.

```
NAME$="Manfred"
```

```
PRINT MID$(NAME$,3,2)
```

Ausgabe:

```
nf
```


Die Zahl 3 gibt die Startposition an.

Die Zahl 2 gibt die Anzahl der Zeichen an.

12.12. Groß- und Kleinbuchstaben

Mit UCASE\$() wird ein Text in Großbuchstaben umgewandelt.

```
NAME$="Manfred"  
  
PRINT UCASE$(NAME$)
```

Ausgabe:

```
MANFRED
```

Mit LCASE\$() wird ein Text in Kleinbuchstaben umgewandelt.

```
NAME$="Manfred"  
  
PRINT LCASE$(NAME$)
```

Ausgabe:

```
manfred
```

12.13. Texte vergleichen

Texte können mit IF verglichen werden.

```
INPUT ANTWORT$  
  
IF ANTWORT$="JA" THEN  
    PRINT "Richtig"  
ENDIF
```

Dabei muss die Schreibweise exakt übereinstimmen.

Die Eingabe

```
ja
```

ist nicht dasselbe wie

JA

12.14. Groß- und Kleinschreibung ignorieren

Oft soll die Schreibweise keine Rolle spielen.

Mit UCASE\$() lässt sich das einfach lösen.

```
INPUT ANTWORT$  
  
IF UCASE$(ANTWORT$)="JA" THEN  
    PRINT "Richtig"  
ENDIF
```

Jetzt werden beispielsweise

JA
Ja
ja
jA

alle akzeptiert.

12.15. In einem Text suchen

Mit INSTR() kann nach einem Zeichen oder einer Zeichenfolge gesucht werden.

```
TEXT$="MMBasic macht Spass"  
  
PRINT INSTR(TEXT$,"macht")
```

Ausgabe:

9

Die Funktion liefert die Position zurück, an der der gesuchte Text beginnt.

Wird der Text nicht gefunden, liefert INSTR() den Wert 0.

```
TEXT$="MMBasic macht Spass"  
  
IF INSTR(TEXT$,"Basic")>0 THEN  
    PRINT "Text gefunden"
```

```
ELSE  
  PRINT "Text nicht gefunden"  
ENDIF
```

12.16. Zahlen und Texte umwandeln

Mit VAL() wird ein Text in eine Zahl umgewandelt.

```
ZAHL=VAL("123")  
  
PRINT ZAHL+1
```

Ausgabe:

124

Mit STR\$() wird umgekehrt eine Zahl in einen Text umgewandelt.

```
TEXT$=STR$(123)  
  
PRINT "Die Zahl lautet ";TEXT$
```

12.17. Datum und Uhrzeit

MMBasic stellt das aktuelle Datum und die aktuelle Uhrzeit als Zeichenketten zur Verfügung.

```
PRINT DATE$  
PRINT TIME$
```

Die genaue Darstellung hängt von der Konfiguration des Systems ab.

Da Datum und Uhrzeit Zeichenketten sind, können sie mit den bekannten String-Funktionen verarbeitet werden.

```
PRINT "Heute ist ";DATE$  
PRINT "Es ist ";TIME$
```

12.18. Begrüßung abhängig von der Uhrzeit

Mit LEFT\$() lässt sich die Stunde aus der Uhrzeit auslesen.

Da LEFT\$() einen Text zurückliefert, wird dieser mit VAL() in eine Zahl umgewandelt.

```
STUNDE=VAL(LEFT$(TIME$,2))
```

```
IF STUNDE<12 THEN  
  PRINT "Guten Morgen!"  
ELSEIF STUNDE<18 THEN  
  PRINT "Guten Tag!"  
ELSE  
  PRINT "Guten Abend!"  
ENDIF
```

12.19. Ein kleines Praxisprogramm

Das folgende Programm verwendet viele der neuen Funktionen.

```
INPUT NAME$  
  
PRINT  
PRINT "Hallo ";NAME$  
PRINT  
  
PRINT "Heute ist ";DATE$  
PRINT "Aktuelle Uhrzeit: ";TIME$  
PRINT  
  
PRINT "Dein Name hat ";LEN(NAME$);" Zeichen."  
  
PRINT "Er beginnt mit ";LEFT$(NAME$,1)  
  
PRINT "Er endet mit ";RIGHT$(NAME$,1)  
  
IF VAL(LEFT$(TIME$,2))<12 THEN  
  PRINT "Guten Morgen!"  
ELSEIF VAL(LEFT$(TIME$,2))<18 THEN  
  PRINT "Guten Tag!"  
ELSE  
  PRINT "Guten Abend!"  
ENDIF
```

12.20. Experimentiere!

Probiere folgende Änderungen aus:

- Gib die ersten drei Zeichen eines Namens aus.
- Gib die letzten drei Zeichen aus.
- Wandle einen Namen in Großbuchstaben um.
- Wandle einen Namen in Kleinbuchstaben um.

- Suche nach einem Wort in einem Text.

12.21. Probier's selbst!

Versuche folgende Aufgaben:

1. Frage den Namen des Benutzers ab.
2. Gib die Anzahl der Zeichen aus.
3. Gib den ersten Buchstaben aus.
4. Gib den letzten Buchstaben aus.
5. Gib den Namen komplett in Großbuchstaben aus.
6. Suche nach einem bestimmten Wort in einem Satz.
7. Gib Datum und Uhrzeit aus.

12.22. Zusammenfassung

In diesem Kapitel hast du gelernt, mit Texten zu arbeiten.

Du kennst nun:

- String-Variablen
- LEN()
- LEFT\$()
- MID\$()
- RIGHT\$()
- UCASE\$()
- LCASE\$()
- INSTR()
- VAL()
- STR\$()
- DATE\$
- TIME\$

Damit kannst du Texte speichern, verändern und auswerten.

Im nächsten Kapitel werden wir das bisher Gelernte in einem größeren Praxisprojekt einsetzen.

Chapter 13. Little Professor

13.1. Das lernst du

In diesem Kapitel verwendest du viele der bisher gelernten Befehle, um ein vollständiges Programm zu entwickeln.

Dabei kommen unter anderem zum Einsatz:

- Variablen
- Zufallszahlen
- Benutzereingaben
- Entscheidungen mit IF
- Schleifen mit FOR
- Punktzählung

Am Ende des Kapitels besitzt du deinen eigenen kleinen Mathematiktrainer.

13.2. Was ist ein Little Professor?

Der Little Professor war ein Lerncomputer von Texas Instruments.

Er stellte Rechenaufgaben und überprüfte die Antworten des Benutzers.

Das Gerät erfreute sich besonders bei Schülern großer Beliebtheit, da es das Kopfrechnen auf spielerische Weise trainierte.

In diesem Kapitel entwickeln wir eine einfache Version mit MMBasic.

13.3. Unsere erste Rechenaufgabe

Beginnen wir mit einer einzigen Aufgabe.

```
A=INT(RND*10)
B=INT(RND*10)

PRINT A;" + ";B;" = ?"

INPUT ANTWORT

IF ANTWORT=A+B THEN
  PRINT "Richtig!"
ELSE
  PRINT "Leider falsch!"
ENDIF
```

13.4. Das Programm verstehen

Die Variablen **A** und **B** erhalten Zufallszahlen zwischen 0 und 9.

```
A=INT(RND*10)
B=INT(RND*10)
```

Anschließend wird die Aufgabe ausgegeben.

```
PRINT A;" + ";B;" = ?"
```

Mit **INPUT** wird die Antwort eingelesen.

Danach vergleicht die IF-Anweisung die Eingabe mit dem korrekten Ergebnis.

13.5. Mehrere Aufgaben hintereinander

Ein echter Little Professor stellt natürlich nicht nur eine einzige Aufgabe.

Mit einer FOR-Schleife können wir mehrere Aufgaben erzeugen.

```
FOR RUNDE=1 TO 10

  A=INT(RND*10)
  B=INT(RND*10)

  PRINT
  PRINT "Aufgabe ";RUNDE
  PRINT A;" + ";B;" = ?"

  INPUT ANTWORT

  IF ANTWORT=A+B THEN
    PRINT "Richtig!"
  ELSE
    PRINT "Leider falsch!"
  ENDIF

NEXT RUNDE
```

Jetzt werden zehn Aufgaben hintereinander gestellt.

13.6. Punkte zählen

Natürlich möchten wir wissen, wie viele Aufgaben richtig gelöst wurden.

Dafür verwenden wir eine Variable mit dem Namen **PUNKTE**.

```
PUNKTE=0
```

Für jede richtige Antwort erhöhen wir den Punktestand um eins.

```
PUNKTE=PUNKTE+1
```

13.7. Die richtige Lösung anzeigen

Wird eine Aufgabe falsch beantwortet, kann das Programm die richtige Lösung anzeigen.

```
IF ANTWORT=A+B THEN
  PRINT "Richtig!"
  PUNKTE=PUNKTE+1
ELSE
  PRINT "Leider falsch!"
  PRINT "Richtig waere ";A+B
ENDIF
```

Dadurch kann der Spieler aus seinen Fehlern lernen.

13.8. Das komplette Programm

```
PUNKTE=0

FOR RUNDE=1 TO 10

  A=INT(RND*10)
  B=INT(RND*10)

  PRINT
  PRINT "Aufgabe ";RUNDE
  PRINT A;" + ";B;" = ?"

  INPUT ANTWORT

  IF ANTWORT=A+B THEN
    PRINT "Richtig!"
    PUNKTE=PUNKTE+1
  ELSE
    PRINT "Leider falsch!"
    PRINT "Richtig waere ";A+B
  ENDIF
```



```
NEXT RUNDE
```

```
PRINT
```

```
PRINT "Du hast ";PUNKTE;" von 10 Punkten erreicht."
```

13.9. Eine Beispielrunde

Eine mögliche Programmausführung könnte so aussehen:

```
Aufgabe 1
```

```
7 + 5 = ?
```

```
12
```

```
Richtig!
```

```
Aufgabe 2
```

```
3 + 8 = ?
```

```
12
```

```
Leider falsch!
```

```
Richtig waere 11
```

```
Aufgabe 3
```

```
6 + 1 = ?
```

```
7
```

```
Richtig!
```

13.10. Das Ergebnis bewerten

Am Ende können wir das Ergebnis bewerten.

```
PRINT
```

```
IF PUNKTE=10 THEN
```

```
  PRINT "Perfekt!"
```

```
ELSEIF PUNKTE>=8 THEN
```

```
  PRINT "Sehr gut!"
```

```
ELSEIF PUNKTE>=5 THEN
```

```
  PRINT "Gut gemacht!"
```

```
ELSE
```

```
  PRINT "Weiter ueben!"
```

```
ENDIF
```

Dadurch erhält der Spieler eine direkte Rückmeldung über seine Leistung.

13.11. Verbesserungsmöglichkeiten

Unser Little Professor funktioniert bereits erstaunlich gut.

Es gibt jedoch viele Möglichkeiten zur Erweiterung:

- größere Zahlenbereiche
- Subtraktionsaufgaben
- Multiplikationsaufgaben
- Division
- verschiedene Schwierigkeitsstufen
- mehrere Spieler

In den nächsten Kapiteln werden wir einige dieser Ideen umsetzen.

13.12. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende Zahlen bis 20.
- Verwende Zahlen bis 100.
- Stelle 20 Aufgaben.
- Vergib zwei Punkte pro richtiger Antwort.
- Ändere die Bewertung am Ende.

13.13. Probier's selbst!

Versuche folgende Aufgaben:

1. Ergänze Subtraktionsaufgaben.
2. Ergänze Multiplikationsaufgaben.
3. Frage den Namen des Spielers ab.
4. Begrüße den Spieler persönlich.
5. Gib die Anzahl der falschen Antworten aus.
6. Zeige am Ende die erreichte Prozentzahl an.

13.14. Zusammenfassung

In diesem Kapitel hast du ein vollständiges Lernprogramm entwickelt.

Dabei wurden viele der bisher gelernten Befehle verwendet:

- PRINT
- INPUT
- IF ... THEN ... ELSE
- FOR ... NEXT

- RND
- Variablen

Du hast gesehen, wie aus vielen kleinen Bausteinen ein nützliches Programm entsteht.

Im nächsten Kapitel werden wir den Little Professor weiter verbessern und um zusätzliche Funktionen erweitern.

Chapter 14. Little Professor Version 2

14.1. Das lernst du

In diesem Kapitel verbessern wir unseren Little Professor aus dem letzten Kapitel.

Dabei lernst du:

- zufällig verschiedene Aufgabentypen auszuwählen
- Addition, Subtraktion, Multiplikation und Division zu kombinieren
- SUB zur Strukturierung eines Programms einzusetzen
- Funktionen und Variablen sinnvoll zusammenzuarbeiten
- den Spieler persönlich anzusprechen

Am Ende des Kapitels besitzt du einen deutlich leistungsfähigeren Mathematiktrainer.

14.2. Unser Programm wächst

Der Little Professor aus dem letzten Kapitel konnte bereits Additionsaufgaben erzeugen.

Nun möchten wir weitere Rechenarten hinzufügen:

- Addition
- Subtraktion
- Multiplikation
- Division

Außerdem soll der Spieler mit seinem Namen begrüßt werden.

14.3. Der Spielername

Zu Beginn fragen wir den Namen des Spielers ab.

```
INPUT "Wie heisst du ";NAME$  
  
PRINT  
PRINT "Hallo ";NAME$  
PRINT
```

14.4. Zufällig eine Rechenart auswählen

Für jede Aufgabe soll eine Rechenart zufällig ausgewählt werden.

Dazu verwenden wir eine Zufallszahl zwischen 0 und 3.

```
OP=INT(RND*4)
```

Die Werte bedeuten:

```
0 = Addition  
1 = Subtraktion  
2 = Multiplikation  
3 = Division
```

14.5. Eine Aufgabe erzeugen

Damit unser Hauptprogramm übersichtlich bleibt, erzeugen wir die Aufgabe in einer SUB.

```
SUB NeueAufgabe  
  
    OP=INT(RND*4)  
  
END SUB
```

Später wird diese SUB alle benötigten Werte erzeugen.

14.6. Additionsaufgaben

```
A=INT(RND*10)  
B=INT(RND*10)  
  
ERG=A+B  
  
PRINT A;" + ";B;" = ?"
```

14.7. Subtraktionsaufgaben

Bei Subtraktionsaufgaben möchten wir negative Ergebnisse vermeiden.

```
A=INT(RND*10)+10  
B=INT(RND*10)  
  
ERG=A-B  
  
PRINT A;" - ";B;" = ?"
```

14.8. Multiplikationsaufgaben

```
A=INT(RND*10)
B=INT(RND*10)

ERG=A*B

PRINT A;" * ";B;" = ?"
```

14.9. Divisionsaufgaben

Divisionen sollen immer ganzzahlig lösbar sein.

Deshalb erzeugen wir zuerst das Ergebnis.

```
ERG=INT(RND*10)+1

B=INT(RND*10)+1

A=ERG*B

PRINT A;" / ";B;" = ?"
```

Dadurch entstehen Aufgaben wie:

```
24 / 6 = ?
```

Die Lösung ist immer eine ganze Zahl.

14.10. Die SUB vervollständigen

Nun fassen wir alle Aufgabentypen zusammen.

```
SUB NeueAufgabe

  OP=INT(RND*4)

  IF OP=0 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A+B

    PRINT A;" + ";B;" = ?"
```

```

ELSEIF OP=1 THEN

    A=INT(RND*10)+10
    B=INT(RND*10)

    ERG=A-B

    PRINT A;" - ";B;" = ?"

ELSEIF OP=2 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A*B

    PRINT A;" * ";B;" = ?"

ELSE

    ERG=INT(RND*10)+1

    B=INT(RND*10)+1

    A=ERG*B

    PRINT A;" / ";B;" = ?"

ENDIF

END SUB

```

14.11. Das komplette Programm

```

INPUT "Wie heisst du ";NAME$

PRINT
PRINT "Hallo ";NAME$
PRINT

PUNKTE=0

FOR RUNDE=1 TO 10

    PRINT
    PRINT "Aufgabe ";RUNDE

    NeueAufgabe

```

```

INPUT ANTWORT

IF ANTWORT=ERG THEN

    PRINT "Richtig!"

    PUNKTE=PUNKTE+1

ELSE

    PRINT "Leider falsch!"

    PRINT "Richtig waere ";ERG

ENDIF

NEXT RUNDE

PRINT
PRINT NAME$;", du hast ";PUNKTE;" von 10 Punkten erreicht."

PROZENT=PUNKTE*100/10

IF PROZENT=100 THEN

    PRINT "Perfekt!"

ELSEIF PROZENT>=90 THEN

    PRINT "Sehr gut!"

ELSEIF PROZENT>=80 THEN

    PRINT "Gut gemacht!"

ELSEIF PROZENT>=60 THEN

    PRINT "Das war ordentlich."

ELSE

    PRINT "Weiter ueben!"

ENDIF

SUB NeueAufgabe

    OP=INT(RND*4)

```



```

IF OP=0 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A+B

    PRINT A;" + ";B;" = ?"

ELSEIF OP=1 THEN

    A=INT(RND*10)+10
    B=INT(RND*10)

    ERG=A-B

    PRINT A;" - ";B;" = ?"

ELSEIF OP=2 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A*B

    PRINT A;" * ";B;" = ?"

ELSE

    ERG=INT(RND*10)+1

    B=INT(RND*10)+1

    A=ERG*B

    PRINT A;" / ";B;" = ?"

ENDIF

END SUB

```

14.12. Beispielausgabe

Wie heisst du? Manfred

Hallo Manfred

Aufgabe 1

$7 + 5 = ?$

12

Richtig!

Aufgabe 2

$24 / 6 = ?$

4

Richtig!

Aufgabe 3

$8 * 7 = ?$

55

Leider falsch!

Richtig waere 56

14.13. Warum SUB hier sinnvoll ist

Die Aufgabe wird in jedem Durchlauf neu erzeugt.

Würden wir den gesamten Code direkt in die Schleife schreiben, würde das Hauptprogramm schnell unübersichtlich werden.

Durch die SUB bleibt der Ablauf klar erkennbar:

NeueAufgabe

INPUT ANTWORT

Antwort prüfen

14.14. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende Zahlen bis 20.
- Stelle 20 Aufgaben.
- Verwende nur Multiplikationsaufgaben.
- Verwende nur Divisionsaufgaben.
- Ändere die Bewertung am Ende.

14.15. Probier's selbst!

Versuche folgende Aufgaben:

1. Füge eine Schwierigkeitsstufe hinzu.

2. Lasse den Spieler die Anzahl der Aufgaben wählen.
3. Vergib Bonuspunkte für besonders schnelle Lösungen.
4. Speichere die Anzahl richtiger und falscher Antworten getrennt.
5. Zeige am Ende die erreichte Prozentzahl an.

14.16. Zusammenfassung

In diesem Kapitel hast du den Little Professor erweitert.

Du hast gelernt:

- verschiedene Rechenarten zu kombinieren
- SUB zur Strukturierung einzusetzen
- Zufallszahlen für unterschiedliche Aufgaben zu verwenden
- den Spieler persönlich anzusprechen
- Ergebnisse zu bewerten

Das Programm ist nun deutlich leistungsfähiger und ähnelt bereits einem echten Lerncomputer.

Im nächsten Kapitel werden wir weitere Verbesserungen einbauen und erstmals Arrays verwenden, um Statistiken über die gelösten Aufgaben zu speichern.

Chapter 15. Little Professor Version 3

15.1. Das lernst du

In diesem Kapitel erweitern wir unseren Little Professor erneut.

Dabei lernst du:

- Arrays in einem größeren Projekt einzusetzen
- Antworten und Lösungen zu speichern
- Ergebnisse auszuwerten
- einfache Statistiken zu erstellen
- dem Spieler gezielt Rückmeldungen zu geben

Am Ende des Kapitels besitzt du einen Mathematiktrainer, der nicht nur Punkte zählt, sondern auch die Fehler des Spielers analysiert.

15.2. Warum noch eine Version?

Unser Little Professor kann bereits:

- verschiedene Rechenarten erzeugen
- Antworten prüfen
- Punkte zählen
- Ergebnisse bewerten

Am Ende kennen wir jedoch nur die erreichte Punktzahl.

Wir wissen nicht:

- Welche Aufgaben falsch beantwortet wurden?
- Welche Antwort der Spieler eingegeben hat?
- Bei welcher Rechenart die meisten Fehler auftraten?

Das möchten wir nun ändern.

15.3. Die Aufgaben speichern

Für jede Aufgabe speichern wir:

- die Aufgabe selbst
- die richtige Lösung
- die Antwort des Spielers

- die Rechenart

Dafür legen wir einige Arrays an.

```
DIM AUFGABE$(9)
DIM LOESUNG(9)
DIM EINGABE(9)
DIM TYP(9)
```

Da wir zehn Aufgaben stellen, benötigen wir die Elemente 0 bis 9.

15.4. Die Arrays füllen

Nach jeder Aufgabe speichern wir die Daten.

```
AUFGABE$(RUNDE-1)=TEXT$
LOESUNG(RUNDE-1)=ERG
EINGABE(RUNDE-1)=ANTWORT
TYP(RUNDE-1)=OP
```

Dadurch stehen die Informationen auch nach dem Spiel noch zur Verfügung.

15.5. Eine Aufgabe als Text speichern

Zusätzlich erzeugen wir einen Text, der die Aufgabe beschreibt.

Bei einer Addition:

```
TEXT$=STR$(A)+" + "+STR$(B)
```

Bei einer Multiplikation:

```
TEXT$=STR$(A)+" * "+STR$(B)
```

Dieser Text wird später für die Auswertung benötigt.

15.6. Das komplette Programm

```
INPUT "Wie heisst du ";NAME$

DIM AUFGABE$(9)
DIM LOESUNG(9)
DIM EINGABE(9)
DIM TYP(9)
```

```

PUNKTE=0

FOR RUNDE=1 TO 10

    NeueAufgabe

    PRINT
    PRINT "Aufgabe ";RUNDE
    PRINT TEXT$;" = ?"

    INPUT ANTWORT

    AUFGABE$(RUNDE-1)=TEXT$
    LOESUNG(RUNDE-1)=ERG
    EINGABE(RUNDE-1)=ANTWORT
    TYP(RUNDE-1)=OP

    IF ANTWORT=ERG THEN

        PRINT "Richtig!"

        PUNKTE=PUNKTE+1

    ELSE

        PRINT "Leider falsch!"

        PRINT "Richtig waere ";ERG

    ENDIF

NEXT RUNDE

PRINT
PRINT NAME$;", du hast ";PUNKTE;" von 10 Punkten erreicht."


PRINT
PRINT "Auswertung"
PRINT "-----"

FOR I=0 TO 9

    IF EINGABE(I)=LOESUNG(I) THEN

        PRINT AUFGABE$(I);" = ";LOESUNG(I);" OK"

    ELSE

```

```

PRINT AUFGABE$(I);" = ";LOESUNG(I)
PRINT "Deine Antwort: ";EINGABE(I)

ENDIF

NEXT I

SUB NeueAufgabe

OP=INT(RND*4)

IF OP=0 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A+B

    TEXT$=STR$(A)+" + "+STR$(B)

ELSEIF OP=1 THEN

    A=INT(RND*10)+10
    B=INT(RND*10)

    ERG=A-B

    TEXT$=STR$(A)+" - "+STR$(B)

ELSEIF OP=2 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A*B

    TEXT$=STR$(A)+" * "+STR$(B)

ELSE

    ERG=INT(RND*10)+1

    B=INT(RND*10)+1

    A=ERG*B

    TEXT$=STR$(A)+" / "+STR$(B)

ENDIF

```

END SUB

15.7. Die Auswertung verstehen

Während des Spiels werden alle Antworten gespeichert.

Nach der letzten Aufgabe durchläuft eine FOR-Schleife die Arrays.

```
FOR I=0 TO 9
  ...
NEXT I
```

Dabei werden die gespeicherten Informationen ausgewertet.

15.8. Beispielausgabe

Manfred, du hast 7 von 10 Punkten erreicht.

Auswertung

$7 + 5 = 12$ OK

$8 * 7 = 56$

Deine Antwort: 55

$24 / 6 = 4$ OK

$15 - 9 = 6$

Deine Antwort: 8

Jetzt erkennt der Spieler sofort, welche Aufgaben falsch beantwortet wurden.

15.9. Fehler nach Rechenart zählen

Mit den gespeicherten Aufgabentypen können wir weitere Statistiken erstellen.

```
ADD=0
SUB=0
MUL=0
DIV=0
```

```
FOR I=0 TO 9
```



```
IF EINGABE(I)<>LOESUNG(I) THEN
```

```
    IF TYP(I)=0 THEN ADD=ADD+1
```

```
    IF TYP(I)=1 THEN SUB=SUB+1
```

```
    IF TYP(I)=2 THEN MUL=MUL+1
```

```
    IF TYP(I)=3 THEN DIV=DIV+1
```

```
ENDIF
```

```
NEXT I
```

Danach können wir die Statistik ausgeben:

```
PRINT
```

```
PRINT "Fehlerstatistik"
```

```
PRINT "Additionen:      ";ADD
```

```
PRINT "Subtraktionen:   ";SUB
```

```
PRINT "Multiplikation:  ";MUL
```

```
PRINT "Divisionen:      ";DIV
```

15.10. Warum Arrays hier wichtig sind

Ohne Arrays würden die Antworten nach jeder Aufgabe verloren gehen.

Durch die Arrays können wir:

- Aufgaben speichern
- Antworten speichern
- Lösungen speichern
- Statistiken berechnen
- Fehler analysieren

Genau für solche Aufgaben werden Arrays in der Praxis verwendet.

15.11. Experimentiere!

Probiere folgende Änderungen aus:

- Erhöhe die Anzahl der Aufgaben auf 20.
- Speichere zusätzlich die benötigte Zeit.
- Gib nur die falsch beantworteten Aufgaben aus.
- Zähle richtige Antworten je Rechenart.

15.12. Probier's selbst!

Versuche folgende Aufgaben:

1. Speichere den Namen des Spielers.
2. Gib die erreichte Prozentzahl aus.
3. Zeige nur die falschen Antworten an.
4. Ermittle die schwierigste Rechenart.
5. Vergib eine Schulnote.

15.13. Zusammenfassung

In diesem Kapitel hast du Arrays in einem echten Projekt eingesetzt.

Du hast gelernt:

- Daten in Arrays zu speichern
- Informationen später auszuwerten
- Statistiken zu berechnen
- Fehler gezielt zu analysieren

Der Little Professor ist damit von einem einfachen Übungsprogramm zu einem echten Lernsystem geworden.

Im nächsten Kapitel werden wir lernen Dateien anzulegen, Daten in Dateien zu speichern und diese wieder auszulesen.

Chapter 16. Dateien speichern und laden

16.1. Das lernst du

In diesem Kapitel lernst du:

- Dateien anzulegen
- Daten in Dateien zu speichern
- Dateien wieder einzulesen
- Textdateien zu verwenden
- Highscores und Einstellungen dauerhaft zu speichern

Am Ende des Kapitels können deine Programme Informationen auch nach dem Programmende behalten.

16.2. Das Problem

Bisher gingen alle Daten verloren, sobald ein Programm beendet wurde.

Nehmen wir unseren Little Professor als Beispiel.

Ein Spieler erreicht 9 von 10 Punkten.

Nach dem nächsten Programmstart ist dieses Ergebnis vergessen.

Es wäre viel besser, wenn wir die Ergebnisse dauerhaft speichern könnten.

Genau dafür gibt es Dateien.

16.3. Was ist eine Datei?

Eine Datei ist ein Bereich auf einem Speichermedium, in dem Informationen dauerhaft gespeichert werden.

Typische Beispiele sind:

- Texte
- Bilder
- Programme
- Tabellen
- Spielstände

Auch unsere BASIC-Programme können Dateien lesen und schreiben.

16.4. Eine Datei öffnen

Bevor eine Datei verwendet werden kann, muss sie geöffnet werden.

Dies geschieht mit dem Befehl OPEN.

```
OPEN "test.txt" FOR OUTPUT AS #1
```

Dabei bedeutet:

Teil	Bedeutung
test.txt	Dateiname
OUTPUT	Datei zum Schreiben öffnen
#1	Dateinummer

Die Dateinummer wird später verwendet, um auf die geöffnete Datei zuzugreifen.

16.5. Eine Datei schreiben

Mit PRINT können Daten in eine Datei geschrieben werden.

```
OPEN "test.txt" FOR OUTPUT AS #1  
  
PRINT #1,"Hallo Welt"  
  
CLOSE #1
```

Nach dem Ausführen enthält die Datei:

```
Hallo Welt
```

16.6. Dateien immer schließen

Nach der Arbeit mit einer Datei sollte diese geschlossen werden.

```
CLOSE #1
```

Dadurch werden alle Daten sicher gespeichert.

16.7. Eine Datei lesen

Zum Lesen wird die Datei im Modus INPUT geöffnet.

```
OPEN "test.txt" FOR INPUT AS #1

LINE INPUT #1,A$

PRINT A$

CLOSE #1
```

Ausgabe:

```
Hallo Welt
```

16.8. LINE INPUT

LINE INPUT liest eine komplette Zeile aus einer Datei.

```
LINE INPUT #1,TEXT$
```

Dies ist besonders praktisch bei Textdateien.

16.9. Mehrere Zeilen speichern

Natürlich können auch mehrere Zeilen geschrieben werden.

```
OPEN "namen.txt" FOR OUTPUT AS #1

PRINT #1,"Manfred"
PRINT #1,"Geoff"
PRINT #1,"Peter"

CLOSE #1
```

Die Datei enthält anschließend:

```
Manfred
Geoff
Peter
```

16.10. Daten anhängen

Normalerweise überschreibt OUTPUT eine vorhandene Datei.

Soll der Inhalt erhalten bleiben, verwendet man APPEND.

```
OPEN "namen.txt" FOR APPEND AS #1  
  
PRINT #1,"Sarah"  
  
CLOSE #1
```

Die neue Zeile wird an das Ende der Datei angehängt.

16.11. Der Unterschied zwischen OUTPUT und APPEND

```
OPEN "daten.txt" FOR OUTPUT AS #1
```

öffnet eine Datei zum Schreiben und löscht einen eventuell vorhandenen Inhalt.

```
OPEN "daten.txt" FOR APPEND AS #1
```

öffnet die Datei zum Anhängen neuer Daten.

Vorhandene Daten bleiben erhalten.

16.12. Mit WRITE speichern

Neben PRINT gibt es auch den Befehl WRITE.

```
OPEN "spieler.txt" FOR OUTPUT AS #1  
  
WRITE #1,"Manfred",9  
  
CLOSE #1
```

Der Inhalt der Datei sieht dann so aus:

```
"Manfred",9
```

WRITE eignet sich besonders gut zum Speichern strukturierter Daten.

16.13. Ein Highscore speichern

Nun speichern wir den Namen eines Spielers und seine Punktzahl.

```
INPUT "Name ";NAME$
```

```
PUNKTE=8

OPEN "highscore.txt" FOR OUTPUT AS #1

WRITE #1,NAME$,PUNKTE

CLOSE #1

PRINT "Highscore gespeichert."
```

16.14. Einen Highscore laden

```
OPEN "highscore.txt" FOR INPUT AS #1

LINE INPUT #1,TEXT$

CLOSE #1

PRINT TEXT$
```

Ausgabe:

```
"Manfred",8
```

16.15. Dateinamen und Verzeichnisse

Dateien können auch in Unterverzeichnissen gespeichert werden.

Beispiel:

```
OPEN "daten\spieler.txt" FOR OUTPUT AS #1
```

Es können lange Dateinamen verwendet werden.

```
OPEN "mein_erster_highscore.txt" FOR OUTPUT AS #1
```

16.16. Wichtige Betriebsarten

MMBasic kennt verschiedene Betriebsarten beim Öffnen einer Datei.

Modus	Bedeutung
INPUT	Datei lesen

Modus	Bedeutung
OUTPUT	Datei schreiben (überschreibt vorhandene Datei)
APPEND	Daten an bestehende Datei anhängen
RANDOM	Datei lesen und schreiben mit direktem Zugriff

Für die meisten Programme reichen INPUT, OUTPUT und APPEND völlig aus.

16.17. Fehlerquellen

Ein häufiger Fehler ist das Lesen einer Datei, die nicht existiert.

```
OPEN "unbekannt.txt" FOR INPUT AS #1
```

In diesem Fall erzeugt MMBasic eine Fehlermeldung.

Ebenso sollte jede geöffnete Datei wieder geschlossen werden.

16.18. Experimentiere!

Probiere folgende Änderungen aus:

- Speichere mehrere Namen in einer Datei.
- Speichere mehrere Punktzahlen.
- Verwende APPEND statt OUTPUT.
- Ändere den Dateinamen.

16.19. Probier's selbst!

Versuche folgende Aufgaben:

1. Speichere deinen Namen in einer Datei.
2. Speichere dein Alter in einer Datei.
3. Lies die Datei wieder ein.
4. Erstelle eine kleine Highscore-Datei.
5. Speichere mehrere Spieler untereinander.

16.20. Zusammenfassung

In diesem Kapitel hast du gelernt, mit Dateien zu arbeiten.

Du kennst nun:

- OPEN

- CLOSE
- PRINT
- WRITE
- LINE INPUT

Damit können deine Programme Informationen dauerhaft speichern.

Im nächsten Kapitel werden wir die grafischen Möglichkeiten des PicoMite kennenlernen und erste Zeichnungen auf dem Bildschirm erzeugen.

Grafik und Benutzeroberflächen

Chapter 17. Die verschiedenen Grafikmodi

17.1. Das lernst du

In diesem Kapitel lernst du:

- wie MMBasic Grafiken auf dem Bildschirm darstellt
- was Pixel sind
- wie Bildschirmkoordinaten funktionieren
- wie die Bildschirmgröße ermittelt werden kann
- welche Informationen MMBasic über den Bildschirm bereitstellt

Am Ende des Kapitels verstehst du die wichtigsten Grundlagen für die Grafikprogrammierung.

17.2. Eine neue Welt

Bisher haben wir unsere Programme hauptsächlich mit Texten erstellt.

Beispielsweise:

```
PRINT "Hallo Welt"
```

oder:

```
INPUT NAME$
```

Der Bildschirm wurde dabei wie ein klassisches Terminal verwendet.

MMBasic kann jedoch deutlich mehr.

Mit wenigen Befehlen lassen sich:

- Linien zeichnen
- Kreise darstellen
- Texte frei positionieren
- Farben verwenden
- grafische Benutzeroberflächen erstellen

Bevor wir damit beginnen, sollten wir einige Grundlagen kennenlernen.

17.3. Was ist ein Pixel?

Jeder Bildschirm besteht aus vielen einzelnen Bildpunkten.

Diese Bildpunkte nennt man Pixel.

Jeder Pixel besitzt eine Position auf dem Bildschirm.

Die Position wird durch zwei Werte beschrieben:

- X = horizontale Position
- Y = vertikale Position

17.4. Das Koordinatensystem

In MMBasic befindet sich der Ursprung in der linken oberen Ecke.

Die Position links oben besitzt die Koordinaten:

```
X = 0  
Y = 0
```

Die X-Koordinate wächst nach rechts.

Die Y-Koordinate wächst nach unten.

(0,0)
+----->
|
|
|
|
v

Wenn später eine Linie oder ein Kreis gezeichnet wird, erfolgt dies immer anhand dieser Koordinaten.

17.5. Die Bildschirmgröße

Nicht jeder Bildschirm besitzt dieselbe Auflösung.

MMBasic stellt deshalb zwei Variablen bereit.

```
PRINT MM.HRES  
PRINT MM.VRES
```

Beispiel:

```
640
```

Dies bedeutet:

- 640 Pixel Breite
- 480 Pixel Höhe

Die Variablen haben folgende Bedeutung:

Variable	Beschreibung
MM.HRES	Horizontale Auflösung (Breite)
MM.VRES	Vertikale Auflösung (Höhe)

17.6. Bildschirminformationen anzeigen

Mit dem folgenden Programm kann die aktuelle Bildschirmgröße ausgegeben werden.

```
CLS

PRINT "Bildschirminformationen"
PRINT

PRINT "Breite : ";MM.HRES
PRINT "Hoehe : ";MM.VRES
```

17.7. Schriftarten

MMBasic besitzt mehrere eingebaute Schriftarten.

Je nach Schriftart unterscheiden sich Breite und Höhe der Zeichen.

Die aktuelle Schriftgröße kann abgefragt werden.

```
PRINT MM.INFO(FONTWIDTH)
PRINT MM.INFO(FONTHEIGHT)
```

Beispiel:

```
8
12
```

Das bedeutet:

- 8 Pixel breit

- 12 Pixel hoch

17.8. Warum ist das wichtig?

Wenn Texte an einer bestimmten Position ausgegeben werden sollen, muss bekannt sein, wie groß die verwendete Schrift ist.

Dadurch können Texte exakt positioniert werden.

17.9. Verschiedene Ausgabegeräte

MMBasic unterstützt verschiedene Grafiksysteme.

Dazu gehören:

- LCD-Displays
- VGA-Monitore
- HDMI-Monitore

Je nach verwendeter Hardware unterscheiden sich:

- Auflösung
- Farbtiefe
- verfügbare Grafikmodi

Die Grafikbefehle selbst funktionieren jedoch weitgehend identisch.

17.10. Farben

Farben werden in MMBasic mit der Funktion RGB() beschrieben.

Beispiele:

```
RGB(RED)
RGB(BLUE)
RGB(GREEN)
RGB(YELLOW)
```

Oder mit numerischen Farbwerten:

```
RGB(255,0,0)
RGB(0,255,0)
RGB(0,0,255)
```

Die genaue Verwendung von Farben werden wir im nächsten Kapitel kennenlernen.

17.11. Ein erstes Informationsprogramm

Das folgende Programm zeigt einige wichtige Informationen über den aktuell verwendeten Bildschirm an.

```
CLS

PRINT "Grafiksystem"
PRINT "-----"
PRINT

PRINT "Breite      : ";MM.HRES
PRINT "Hoehe       : ";MM.VRES

PRINT
PRINT "Fontbreite   : ";MM.INFO(FONTWIDTH)
PRINT "Fonthoehe    : ";MM.INFO(FONTHEIGHT)
```

Eine mögliche Ausgabe könnte so aussehen:

```
Grafiksystem
-----

Breite      : 640
Hoehe       : 480

Fontbreite   : 8
Fonthoehe    : 12
```

17.12. Experimentiere!

Probiere folgende Änderungen aus:

- Führe das Programm auf verschiedenen Geräten aus.
- Vergleiche HDMI und VGA.
- Ändere die Schriftart und beobachte die Werte.
- Notiere die Bildschirmgröße deines Systems.

17.13. Probier's selbst!

Versuche folgende Aufgaben:

1. Gib die Bildschirmbreite aus.
2. Gib die Bildschirmhöhe aus.
3. Gib die Fontbreite aus.

4. Gib die Fonthöhe aus.
5. Berechne die Bildschirmmitte.

17.14. Zusammenfassung

In diesem Kapitel hast du die Grundlagen der Grafikprogrammierung kennengelernt.

Du weißt nun:

- was Pixel sind
- wie das Koordinatensystem funktioniert
- wie die Bildschirmgröße bestimmt wird
- wie Schriftgrößen abgefragt werden können
- wie Farben beschrieben werden

Im nächsten Kapitel werden wir die ersten grafischen Objekte auf dem Bildschirm zeichnen.

Chapter 18. Farben und Schriftarten

18.1. Das lernst du

In diesem Kapitel lernst du:

- Farben in MMBasic zu verwenden
- die Funktion RGB() einzusetzen
- Vorder- und Hintergrundfarben festzulegen
- verschiedene Schriftarten auszuwählen
- Schriftgrößen zu verändern

Am Ende des Kapitels kannst du farbige Bildschirmausgaben gestalten und die passende Schriftart auswählen.

18.2. Farben in MMBasic

MMBasic verwendet ein modernes Farbsystem mit 24 Bit Farbtiefe.

Jede Farbe setzt sich aus den Anteilen für:

- Rot
- Grün
- Blau

zusammen.

Daher stammt auch die Bezeichnung RGB.

18.3. Farben mit RGB() erzeugen

Die Funktion RGB() erwartet drei Werte zwischen 0 und 255.

```
RGB(Rot, Gruen, Blau)
```

Beispiele:

```
RGB(255,0,0)  
RGB(0,255,0)  
RGB(0,0,255)
```

erzeugen:

- Rot

- Grün
- Blau

18.4. Farbkonstanten verwenden

Häufig verwendete Farben können direkt über ihren Namen angegeben werden.

```
RGB(RED)
RGB(GREEN)
RGB(BLUE)
RGB(YELLOW)
RGB(CYAN)
RGB(MAGENTA)
RGB(WHITE)
RGB(BLACK)
```

Dadurch werden Programme leichter lesbar.

18.5. Vorder- und Hintergrundfarbe

Mit dem Befehl COLOUR werden die Standardfarben festgelegt.

```
COLOUR RGB(YELLOW), RGB(BLUE)
```

Dabei bedeutet:

- Gelb = Vordergrundfarbe
- Blau = Hintergrundfarbe

Alle folgenden Textausgaben verwenden diese Farben.

```
CLS

COLOUR RGB(YELLOW), RGB(BLUE)

PRINT "Hallo MMBasic"
```

18.6. Der Bildschirmhintergrund

Mit CLS wird der Bildschirm gelöscht.

Ohne Parameter wird die aktuelle Hintergrundfarbe verwendet.

```
CLS
```

Es kann aber auch direkt eine Farbe angegeben werden.

```
CLS RGB(BLACK)
```

18.7. Ein kleines Farbexperiment

```
CLS RGB(BLACK)

COLOUR RGB(YELLOW), RGB(BLACK)

PRINT "Gelber Text"

COLOUR RGB(CYAN), RGB(BLACK)

PRINT "Cyanfarbener Text"

COLOUR RGB(RED), RGB(BLACK)

PRINT "Roter Text"
```

Probiere verschiedene Farbkombinationen aus.

18.8. Schriftarten

MMBasic enthält bereits mehrere eingebaute Schriftarten.

Jede Schrift besitzt eine andere Größe.

Die Standard-Schrift ist Schriftart 1.

18.9. Schriftart auswählen

Mit FONT wird die gewünschte Schriftart ausgewählt.

```
FONT 1
PRINT "Schriftart 1"

FONT 2
PRINT "Schriftart 2"

FONT 3
PRINT "Schriftart 3"
```

18.10. Die wichtigsten Schriftarten

Font	Beschreibung
1	Standardschrift
2	Mittlere Schrift
3	Große Schrift
5	Sehr große Schrift
6	Große Ziffern

18.11. Schriftgröße ausprobieren

```
CLS
```

```
FONT 1  
PRINT "Kleine Schrift"
```

```
FONT 2  
PRINT "Mittlere Schrift"
```

```
FONT 3  
PRINT "Grosse Schrift"
```

```
FONT 5  
PRINT "Sehr grosse Schrift"
```

18.12. Große Zahlen darstellen

Für Messwerte oder Uhren eignet sich Font 6 besonders gut.

```
CLS
```

```
FONT 6  
  
PRINT "12:45"
```

18.13. Informationen zur aktuellen Schrift

Die Breite und Höhe der aktuellen Schrift können abgefragt werden.

```
PRINT MM.INFO(FONTWIDTH)  
PRINT MM.INFO(FONTHEIGHT)
```

Beispiel:

```
16  
24
```

Die Werte hängen von der gewählten Schriftart und vom verwendeten Grafikmodus ab.

18.14. Ein digitales Display

Das folgende Programm simuliert eine einfache Digitalanzeige.

```
CLS RGB(BLACK)  
  
COLOUR RGB(GREEN), RGB(BLACK)  
  
FONT 6  
  
PRINT TIME$
```

Je nach System wird die aktuelle Uhrzeit in großen Ziffern dargestellt.

18.15. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere Farben.
- Teste verschiedene Schriftarten.
- Kombiniere unterschiedliche Vorder- und Hintergrundfarben.
- Zeige das aktuelle Datum an.

18.16. Probier's selbst!

Versuche folgende Aufgaben:

1. Gib deinen Namen in einer großen Schrift aus.
2. Zeige Datum und Uhrzeit gleichzeitig an.
3. Verwende eine andere Farbe für jede Zeile.
4. Erstelle eine einfache digitale Uhr.

18.17. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Farben mit RGB() zu verwenden

- Vorder- und Hintergrundfarben festzulegen
- Schriftarten auszuwählen
- Informationen über die aktuelle Schrift abzurufen

Im nächsten Kapitel werden wir die ersten grafischen Objekte auf dem Bildschirm zeichnen und lernen PIXEL, LINE und BOX kennen.

Chapter 19. Die ersten Grafikbefehle

19.1. Das lernst du

In diesem Kapitel lernst du:

- den Bildschirm zu löschen
- einzelne Pixel zu setzen
- Linien zu zeichnen
- Koordinaten zu verwenden

Am Ende des Kapitels kannst du eigene einfache Zeichnungen erstellen.

19.2. Endlich wird gezeichnet

In den bisherigen Kapiteln haben wir hauptsächlich mit Texten gearbeitet.

Nun beginnen wir mit der eigentlichen Grafikprogrammierung.

Die wichtigsten Befehle für den Einstieg sind:

- CLS
- PIXEL
- LINE

19.3. Den Bildschirm löschen

Mit CLS wird der Bildschirm gelöscht.

```
CLS
```

Der gesamte Bildschirm wird dabei mit der aktuellen Hintergrundfarbe gefüllt.

Es kann auch direkt eine Farbe angegeben werden.

```
CLS RGB(BLACK)
```

19.4. Ein einzelner Pixel

Der einfachste Grafikbefehl ist PIXEL.

```
PIXEL 100,100
```

Dieser Befehl setzt einen einzelnen Bildpunkt an der Position X=100 und Y=100.

19.5. Ein farbiger Pixel

Natürlich kann auch eine Farbe angegeben werden.

```
PIXEL 100,100,RGB(RED)
```

19.6. Unser erstes Grafikprogramm

```
CLS RGB(BLACK)
```

```
PIXEL 100,100,RGB(RED)
```

Das Ergebnis ist ein einzelner roter Punkt auf schwarzem Hintergrund.

19.7. Mehrere Pixel zeichnen

```
CLS RGB(BLACK)
```

```
PIXEL 100,100,RGB(RED)
```

```
PIXEL 101,100,RGB(RED)
```

```
PIXEL 102,100,RGB(RED)
```

```
PIXEL 103,100,RGB(RED)
```

```
PIXEL 104,100,RGB(RED)
```

Aus einzelnen Pixeln entstehen später Linien, Kreise und Bilder.

19.8. Linien zeichnen

Mit LINE wird eine Linie zwischen zwei Punkten gezeichnet.

```
LINE X1,Y1,X2,Y2
```

Beispiel:

```
LINE 10,10,200,10
```

19.9. Horizontale Linien

```
CLS
```



```
LINE 10,50,200,50
```

Die Linie beginnt bei X=10 und endet bei X=200.

Die Y-Koordinate bleibt konstant.

19.10. Vertikale Linien

```
CLS
```

```
LINE 100,20,100,200
```

Diesmal bleibt die X-Koordinate konstant.

19.11. Diagonale Linien

```
CLS
```

```
LINE 10,10,200,150
```

Nun werden sowohl X als auch Y verändert.

19.12. Linienfarben

Auch für Linien kann eine Farbe angegeben werden.

```
LINE 10,10,200,100,,RGB(RED)
```

19.13. Mehrere Linien kombinieren

Mit mehreren Linien können einfache Formen erzeugt werden.

```
CLS
```

```
LINE 50,100,250,100
```

```
LINE 150,20,150,180
```

Das Ergebnis ist ein Kreuz.

19.14. Ein Dreieck zeichnen

CLS

```
LINE 100,50,50,150  
LINE 50,150,150,150  
LINE 150,150,100,50
```

19.15. Ein Haus zeichnen

Mit einigen wenigen Linien lässt sich bereits ein kleines Haus darstellen.

CLS

```
LINE 50,150,150,150  
LINE 150,150,150,80  
LINE 150,80,50,80  
LINE 50,80,50,150  
  
LINE 50,80,100,30  
LINE 100,30,150,80
```

19.16. Das Haus vom Nikolaus

Eine bekannte Zeichenübung ist das Haus vom Nikolaus.

Versuche, es selbst mit LINE zu zeichnen.



19.17. Bildschirmmitte berechnen

Mit MM.HRES und MM.VRES lässt sich die Bildschirmmitte berechnen.

```
MITTEX=MM.HRES/2  
MITTEY=MM.VRES/2
```

```
PIXEL MITTEX,MITTEY,RGB(RED)
```

Dadurch funktioniert das Programm auf unterschiedlichen Bildschirmgrößen.

19.18. Ein Fadenkreuz zeichnen

```
CLS  
  
LINE MM.HRES/2,0,MM.HRES/2,MM.VRES  
  
LINE 0,MM.VRES/2,MM.HRES,MM.VRES/2
```

Dieses Programm zeichnet ein Fadenkreuz durch die Bildschirmmitte.

19.19. Ein kleines Grafik-Testprogramm

Zum Abschluss verbinden wir die bisher kennengelernten Grafikbefehle mit Schleifen, Zufallszahlen und Farben.

Das folgende Programm zeichnet nacheinander:

- 1000 zufällig verteilte Punkte
- 1000 Kreise
- 1000 Rechtecke

Position, Größe und Farbe werden jeweils zufällig bestimmt.

```
CLS RGB(BLACK)  
  
PRINT "1000 zufaellige Punkte"  
  
FOR I=1 TO 1000  
  
    X=INT(RND*MM.HRES)  
    Y=INT(RND*MM.VRES)  
  
    FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))  
  
    PIXEL X,Y,FARBE  
  
NEXT I  
  
PAUSE 1000  
  
PRINT "1000 zufaellige Kreise"
```

```

FOR I=1 TO 1000

    RADIUS=INT(RND*20)+2

    X=INT(RND*(MM.HRES-2*RADIUS))+RADIUS
    Y=INT(RND*(MM.VRES-2*RADIUS))+RADIUS

    FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))

    CIRCLE X,Y,RADIUS,1,1,FARBE

NEXT I

PAUSE 1000

PRINT "1000 zufaellige Rechtecke"

FOR I=1 TO 1000

    BREITE=INT(RND*50)+5
    HOEHE=INT(RND*50)+5

    X=INT(RND*(MM.HRES-BREITE))
    Y=INT(RND*(MM.VRES-HOEHE))

    FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))

    BOX X,Y,BREITE,HOEHE,1,FARBE

NEXT I

```

19.20. Das Programm verstehen

Für jedes Grafikelement werden zunächst zufällige Bildschirmkoordinaten berechnet.

```

X=INT(RND*MM.HRES)
Y=INT(RND*MM.VRES)

```

Dadurch werden nur Positionen innerhalb der aktuellen Bildschirmauflösung erzeugt.

Auch die Farbe wird zufällig zusammengesetzt:

```

FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))

```

Jeder der drei Farbanteile Rot, Grün und Blau erhält einen zufälligen Wert zwischen 0 und 255.

Bei Kreisen wird zusätzlich ein zufälliger Radius bestimmt:

```
RADIUS=INT(RND*20)+2
```

Die Berechnung der Position berücksichtigt den Radius. Dadurch wird verhindert, dass der Mittelpunkt eines Kreises zu nahe am Bildschirmrand liegt.

Bei den Rechtecken werden Breite und Höhe ebenfalls zufällig erzeugt:

```
BREITE=INT(RND*50)+5  
HOEHE=INT(RND*50)+5
```

Die beiden **PAUSE**-Anweisungen sorgen für eine kurze Unterbrechung zwischen den drei Abschnitten.



Da die Grafikelemente nicht gelöscht werden, überlagern sich Punkte, Kreise und Rechtecke zunehmend.

Möchtest du jeden Test auf einem leeren Bildschirm sehen, füge vor dem jeweiligen Abschnitt eine weitere Anweisung ein:

```
CLS RGB(BLACK)
```

19.21. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere Farben.
- Zeichne ein größeres Haus.
- Zeichne mehrere Dreiecke.
- Zeichne ein Gitter aus Linien.

19.22. Probier's selbst!

Versuche folgende Aufgaben:

1. Zeichne ein Quadrat aus vier Linien.
2. Zeichne einen Stern.
3. Zeichne ein eigenes Haus.
4. Zeichne ein Spielfeld für Tic-Tac-Toe.
5. Zeichne die Umrisse eines Briefumschlags.

19.23. Zusammenfassung

In diesem Kapitel hast du die ersten Grafikbefehle kennengelernt.

Du kennst nun:

- CLS
- PIXEL
- LINE

Außerdem hast du gelernt, mit Bildschirmkoordinaten zu arbeiten.

Im nächsten Kapitel werden wir weitere Grafikbefehle kennenlernen und Rechtecke, Kreise sowie frei positionierbare Texte zeichnen.

Chapter 20. Rechtecke, Kreise und Texte

20.1. Das lernst du

In diesem Kapitel lernst du:

- Rechtecke mit **BOX** zu zeichnen
- abgerundete Rechtecke mit **RBOX** zu erzeugen
- Kreise und Ellipsen darzustellen
- Kreisbögen mit **ARC** zu zeichnen
- Texte frei auf dem Bildschirm zu positionieren
- Formen mit Farben zu füllen

Am Ende des Kapitels kannst du bereits kleine grafische Bildschirmseiten gestalten.

20.2. Mehr als Punkte und Linien

Im letzten Kapitel haben wir einzelne Pixel gesetzt und Linien gezeichnet.

MMBasic bietet weitere Grafikbefehle, mit denen sich häufig benötigte Formen direkt erzeugen lassen:

- **BOX**
- **RBOX**
- **CIRCLE**
- **ARC**
- **TEXT**

Dadurch müssen Rechtecke, Kreise und Texte nicht mühsam aus einzelnen Pixeln zusammengesetzt werden.

20.3. Ein Rechteck zeichnen

Mit **BOX** wird ein Rechteck gezeichnet.

Die grundlegende Schreibweise lautet:

```
BOX X,Y,BREITE,HOEHE
```

Ein Beispiel:

```
CLS RGB(BLACK)
```

```
BOX 50,40,200,100
```

Das Rechteck beginnt an der Position $X=50$, $Y=40$.

Es ist 200 Pixel breit und 100 Pixel hoch.

20.4. Die Parameter von BOX

Der vollständige Befehl lautet:

```
BOX X,Y,BREITE,HOEHE,LINIENBREITE,FARBE,FUELLFARBE
```

Die Parameter haben folgende Bedeutung:

Parameter	Bedeutung
X	Horizontale Position der linken oberen Ecke
Y	Vertikale Position der linken oberen Ecke
BREITE	Breite des Rechtecks in Pixeln
HOEHE	Höhe des Rechtecks in Pixeln
LINIENBREITE	Breite des Rahmens
FARBE	Farbe des Rahmens
FUELLFARBE	Farbe der Fläche

20.5. Ein farbiges Rechteck

```
CLS RGB(BLACK)
```

```
BOX 50,40,200,100,3,RGB(YELLOW),RGB(BLUE)
```

Das Rechteck besitzt nun:

- einen drei Pixel breiten gelben Rahmen
- eine blaue Füllung

20.6. Ein Rechteck ohne Füllung

Soll das Rechteck nicht gefüllt werden, kann die Füllfarbe weggelassen werden.

```
BOX 50,40,200,100,3,RGB(YELLOW)
```

Der Hintergrund innerhalb des Rechtecks bleibt unverändert.

20.7. Mehrere Rechtecke zeichnen

```
CLS RGB(BLACK)

BOX 20,20,100,60,2,RGB(RED)
BOX 50,50,100,60,2,RGB(GREEN)
BOX 80,80,100,60,2,RGB(BLUE)
```

Da sich die Rechtecke überlagern, entsteht bereits eine kleine grafische Komposition.

20.8. Abgerundete Rechtecke

Mit **RBOX** werden Rechtecke mit abgerundeten Ecken gezeichnet.

```
RBOX X,Y,BREITE,HOEHE,RADIUS,FARBE,FUELLFARBE
```

Beispiel:

```
CLS RGB(BLACK)

RBOX 50,50,220,80,15,RGB(WHITE),RGB(BLUE)
```

Der Wert **15** bestimmt den Radius der abgerundeten Ecken.

20.9. Ein grafischer Schalter

Abgerundete Rechtecke eignen sich gut für Schaltflächen.

```
CLS RGB(BLACK)

RBOX 100,80,200,70,12,RGB(WHITE),RGB(GREEN)

TEXT 200,115,"START","CM"
```

Der Text wird in der Mitte der Schaltfläche ausgegeben.

20.10. Einen Kreis zeichnen

Mit **CIRCLE** wird ein Kreis gezeichnet.

Die grundlegende Schreibweise lautet:

```
CIRCLE X,Y,RADIUS
```

Dabei geben **X** und **Y** den Mittelpunkt des Kreises an.

Beispiel:

```
CLS RGB(BLACK)

CIRCLE 160,120,50
```

Der Kreis besitzt:

- Mittelpunkt **X=160**, **Y=120**
- Radius 50 Pixel

20.11. Einen farbigen Kreis zeichnen

```
CLS RGB(BLACK)

CIRCLE 160,120,50,3,1,RGB(YELLOW),RGB(BLUE)
```

Dabei bedeuten die zusätzlichen Parameter:

Parameter	Bedeutung
3	Breite der Kreislinie
1	Seitenverhältnis
RGB(YELLOW)	Farbe der Kreislinie
RGB(BLUE)	Füllfarbe

20.12. Ellipsen zeichnen

Der Parameter für das Seitenverhältnis bestimmt, ob ein Kreis oder eine Ellipse gezeichnet wird.

Ein Wert von **1** erzeugt einen Kreis.

```
CIRCLE 160,120,60,2,1,RGB(WHITE)
```

Ein Wert kleiner als **1** macht die Form schmaler.

```
CIRCLE 160,120,60,2,0.5,RGB(YELLOW)
```

Dadurch entsteht eine Ellipse.

20.13. Ein einfaches Ziel

Mit mehreren Kreisen lassen sich Zielscheiben erzeugen.

```
CLS RGB(BLACK)

X=MM.HRES/2
Y=MM.VRES/2

CIRCLE X,Y,100,2,1,RGB(WHITE)
CIRCLE X,Y,75,2,1,RGB(YELLOW)
CIRCLE X,Y,50,2,1,RGB(RED)
CIRCLE X,Y,25,2,1,RGB(GREEN)
```

Die Position wird aus der Bildschirmgröße berechnet.

Dadurch liegt die Zielscheibe unabhängig von der Auflösung in der Bildschirmmitte.

20.14. Kreisbögen zeichnen

Mit **ARC** können Teile eines Kreises gezeichnet werden.

Die grundlegende Schreibweise lautet:

```
ARC X,Y,INNENRADIUS,AUSSENRADIUS,STARTWINKEL,ENDWINKEL,FARBE
```

Beispiel:

```
CLS RGB(BLACK)

ARC 160,120,45,55,0,180,RGB(YELLOW)
```

Der Winkel 0 befindet sich bei MMBasic an der Zwölf-Uhr-Position.

Die Winkel werden in Grad angegeben.

20.15. Mehrere Kreisbögen kombinieren

```
CLS RGB(BLACK)

X=MM.HRES/2
Y=MM.VRES/2

ARC X,Y,40,50,0,90,RGB(RED)
ARC X,Y,40,50,90,180,RGB(GREEN)
```

```
ARC X,Y,40,50,180,270,RGB(BLUE)
ARC X,Y,40,50,270,360,RGB(YELLOW)
```

Damit entsteht ein Ring aus vier verschiedenfarbigen Abschnitten.

20.16. Texte frei positionieren

Mit **PRINT** wird Text normalerweise an der aktuellen Schreibposition ausgegeben.

Der Befehl **TEXT** ermöglicht dagegen eine pixelgenaue Positionierung.

Die grundlegende Schreibweise lautet:

```
TEXT X,Y,"Text"
```

Beispiel:

```
CLS RGB(BLACK)

TEXT 50,40,"Hallo MMBasic"
```

20.17. Text ausrichten

Der Text kann relativ zu seiner Position ausgerichtet werden.

```
TEXT X,Y,"Text","AUSRICHTUNG"
```

Für die horizontale Ausrichtung stehen zur Verfügung:

Buchstabe	Bedeutung
L	Linksbündig
C	Zentriert
R	Rechtsbündig

Für die vertikale Ausrichtung stehen zur Verfügung:

Buchstabe	Bedeutung
T	Oben
M	Mittig
B	Unten

20.18. Text in der Bildschirmmitte

```
CLS RGB(BLACK)
```

```
TEXT MM.HRES/2,MM.VRES/2,"Hallo MMBasic","CM"
```

C richtet den Text horizontal zentriert aus.

M richtet ihn vertikal mittig aus.

20.19. Schriftart und Skalierung

Mit weiteren Parametern können Schriftart und Vergrößerung festgelegt werden.

```
TEXT 100,80,"MMBasic","LT",2,2
```

Dabei bedeutet:

- Schriftart 2
- Skalierungsfaktor 2

20.20. Farbiger Text

Auch Vorder- und Hintergrundfarbe können direkt angegeben werden.

```
CLS RGB(BLACK)
```

```
TEXT 100,80,"MMBasic","LT",2,2,RGB(YELLOW),RGB(BLUE)
```

20.21. Transparenter Hintergrund

Auf unterstützten Bildschirmen kann der Hintergrund eines Textes transparent bleiben.

Dazu wird als Hintergrundfarbe **-1** angegeben.

```
TEXT 100,80,"MMBasic","LT",2,2,RGB(YELLOW),-1
```

Der bereits vorhandene Bildschirminhalt bleibt zwischen den Buchstaben sichtbar.



Transparenter Text wird nicht von jeder Display-Konfiguration unterstützt.

Auf VGA- und HDMI-Systemen steht diese Möglichkeit in der Regel zur Verfügung.

20.22. Text drehen

Ein dritter Buchstabe in der Ausrichtung legt die Orientierung des Textes fest.

Buchstabe	Ausrichtung
N	Normal
V	Buchstaben senkrecht untereinander
I	Um 180 Grad gedreht
U	Um 90 Grad gegen den Uhrzeigersinn gedreht
D	Um 90 Grad im Uhrzeigersinn gedreht

Beispiel:

```
CLS RGB(BLACK)

TEXT 20,MM.VRES/2,"MMBasic","LMV",2
```

Der Text wird am linken Bildschirmrand senkrecht dargestellt.

20.23. Eine einfache Informationskarte

Nun kombinieren wir Rechteck und Text.

```
CLS RGB(BLACK)

X=MM.HRES/2-150
Y=MM.VRES/2-70

RBOX X,Y,300,140,15,RGB(WHITE),RGB(BLUE)

TEXT MM.HRES/2,Y+35,"MMBasic","CM",2,1,RGB(YELLOW),RGB(BLUE)

TEXT MM.HRES/2,Y+75,DATE$,"CM",1,1,RGB(WHITE),RGB(BLUE)

TEXT MM.HRES/2,Y+105,TIME$,"CM",1,1,RGB(CYAN),RGB(BLUE)
```

Das Ergebnis ist eine kleine Informationskarte mit:

- Überschrift
- Datum
- Uhrzeit

20.24. Ein einfaches Gesicht

Mit Kreisen, Bögen und Linien lässt sich ein Gesicht zeichnen.

```
CLS RGB(BLACK)

X=MM.HRES/2
Y=MM.VRES/2

CIRCLE X,Y,100,3,1,RGB(YELLOW)

CIRCLE X-35,Y-30,10,1,1,RGB(WHITE),RGB(WHITE)
CIRCLE X+35,Y-30,10,1,1,RGB(WHITE),RGB(WHITE)

ARC X,Y+10,45,50,120,240,RGB(YELLOW)
```

Probiere aus, wie sich das Gesicht verändert, wenn du die Positionen oder Winkel veränderst.

20.25. Experimentiere!

Probiere folgende Änderungen aus:

- Zeichne Rechtecke mit verschiedenen Rahmenbreiten.
- Verwende unterschiedliche Füllfarben.
- Erzeuge Kreise mit verschiedenen Radien.
- Zeichne Ellipsen mit unterschiedlichen Seitenverhältnissen.
- Positioniere Texte an verschiedenen Bildschirmrändern.
- Drehe einen Text in alle verfügbaren Richtungen.

20.26. Probier's selbst!

Versuche folgende Aufgaben:

1. Zeichne eine Ampel aus einem Rechteck und drei Kreisen.
2. Gestalte eine Schaltfläche mit **RBOX** und **TEXT**.
3. Zeichne eine Zielscheibe.
4. Erzeuge eine digitale Uhr mit Rahmen.
5. Zeichne ein einfaches Gesicht.
6. Erstelle eine kleine Visitenkarte mit deinem Namen.

20.27. Zusammenfassung

In diesem Kapitel hast du weitere wichtige Grafikbefehle kennengelernt.

Du kannst nun:

- Rechtecke mit **BOX** zeichnen
- abgerundete Rechtecke mit **RBOX** erzeugen
- Kreise und Ellipsen mit **CIRCLE** darstellen
- Kreisbögen mit **ARC** zeichnen
- Texte mit **TEXT** frei positionieren
- Formen mit Farben füllen

Im nächsten Kapitel verbinden wir die Grafikbefehle zu einem vollständigen kleinen Grafikprojekt.

Chapter 21. Treffe die Scheibe

21.1. Das lernst du

In diesem Kapitel lernst du:

- grafische Objekte zu kombinieren
- Bildschirmpositionen zu berechnen
- Entfernungen zu bestimmen
- Funktionen in einem echten Spiel einzusetzen
- ein vollständiges Grafikspiel zu programmieren

Am Ende des Kapitels hast du dein erstes eigenes Grafikspiel entwickelt.

21.2. Unser erstes Grafikspiel

In den letzten Kapiteln haben wir gelernt:

- Linien zu zeichnen
- Kreise darzustellen
- Texte zu positionieren
- Farben zu verwenden

Nun wird es Zeit, alles miteinander zu verbinden.

Wir programmieren ein kleines Geschicklichkeitsspiel.

Der Computer versteckt eine Zielscheibe an einer zufälligen Position auf dem Bildschirm.

Deine Aufgabe ist es, die Position möglichst genau zu erraten.

21.3. So funktioniert das Spiel

Nach dem Start erscheint eine Zielscheibe.

Anschließend gibst du einen Schuss ab.

Dazu werden die Koordinaten eingegeben:

```
X ?  
Y ?
```

Das Programm berechnet anschließend die Entfernung zum Mittelpunkt der Zielscheibe.

Je näher dein Treffer liegt, desto mehr Punkte erhältst du.

21.4. Die Zielscheibe zeichnen

Zunächst benötigen wir eine Funktion zum Zeichnen der Zielscheibe.

```
SUB ZeichneScheibe(X,Y)

    CIRCLE X,Y,60,2,1,RGB(WHITE)

    CIRCLE X,Y,40,2,1,RGB(WHITE)

    CIRCLE X,Y,20,2,1,RGB(WHITE)

END SUB
```

Die Zielscheibe besteht aus drei konzentrischen Kreisen.

21.5. Die Entfernung berechnen

Um die Entfernung zwischen zwei Punkten zu bestimmen, verwenden wir die Formel:

```
Abstand =
Quadratwurzel(
(X1-X2)^2 +
(Y1-Y2)^2 )
```

MMBasic besitzt dafür die Funktion SQR().

21.6. Das komplette Spiel

```
MODE 2

CLS RGB(BLACK)

TX=INT(RND*(MM.HRES-120))+60
TY=INT(RND*(MM.VRES-120))+60

ZeichneScheibe TX,TY

PRINT
PRINT "Treffe die Scheibe!"
PRINT

INPUT "X ";SX
INPUT "Y ";SY

DIST=SQR((SX-TX)^2+(SY-TY)^2)
```

```
CIRCLE SX,SY,4,1,1,RGB(RED),RGB(RED)
```

```
PRINT
```

```
PRINT "Abstand: ";INT(DIST);" Pixel"
```

```
IF DIST<10 THEN
```

```
    PRINT "Volltreffer!"
```

```
ELSEIF DIST<25 THEN
```

```
    PRINT "Sehr gut!"
```

```
ELSEIF DIST<50 THEN
```

```
    PRINT "Gut getroffen!"
```

```
ELSE
```

```
    PRINT "Daneben!"
```

```
ENDIF
```

```
SUB ZeichneScheibe(X,Y)
```

```
    CIRCLE X,Y,60,2,1,RGB(WHITE)
```

```
    CIRCLE X,Y,40,2,1,RGB(WHITE)
```

```
    CIRCLE X,Y,20,2,1,RGB(WHITE)
```

```
END SUB
```

21.7. Das Spiel ausprobieren

Nach dem Programmstart erscheint beispielsweise:

```
Treffe die Scheibe!
```

```
X ? 120
```

```
Y ? 100
```

Anschließend wird dein Schuss als roter Punkt dargestellt.

21.8. Die Bildschirmmitte verwenden

Statt einer zufälligen Position kann die Zielscheibe auch in der Mitte erscheinen.

```
TX=MM.HRES/2  
TY=MM.VRES/2
```

Dadurch lässt sich das Spiel leichter testen.

21.9. Mehrere Schüsse erlauben

Mit einer Schleife können mehrere Versuche durchgeführt werden.

```
FOR RUNDE=1 TO 5  
  
    ...  
  
NEXT RUNDE
```

Dadurch entsteht ein deutlich spannenderes Spiel.

21.10. Punkte vergeben

Die Entfernung kann direkt in Punkte umgerechnet werden.

```
PUNKTE=100-DIST  
  
IF PUNKTE<0 THEN  
    PUNKTE=0  
ENDIF
```

Ein perfekter Treffer bringt 100 Punkte.

21.11. Eine farbige Zielscheibe

Mit Füllfarben wird die Zielscheibe attraktiver.

```
CIRCLE X,Y,60,1,1,RGB(WHITE),RGB(BLUE)  
  
CIRCLE X,Y,40,1,1,RGB(WHITE),RGB(RED)  
  
CIRCLE X,Y,20,1,1,RGB(WHITE),RGB(YELLOW)
```

Jetzt erinnert die Scheibe an eine echte Zielscheibe.

21.12. Highscore speichern

Mit dem Wissen aus Kapitel 16 könntest du die höchste Punktzahl sogar dauerhaft speichern.

```
OPEN "highscore.txt" FOR OUTPUT AS #1

PRINT #1,PUNKTE

CLOSE #1
```

21.13. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere Farben.
- Vergrößere die Zielscheibe.
- Erlaube mehrere Schüsse.
- Vergib Punkte.
- Speichere einen Highscore.

21.14. Probier's selbst!

Versuche folgende Erweiterungen:

1. Führe zehn Schüsse durch.
2. Berechne die Gesamtpunktzahl.
3. Speichere den besten Spieler.
4. Erzeuge unterschiedlich große Zielscheiben.
5. Lasse die Zielscheibe nach jedem Treffer an einer neuen Position erscheinen.

21.15. Zusammenfassung

In diesem Kapitel hast du dein erstes Grafikspiel programmiert.

Dabei kamen viele Techniken zusammen:

- Zufallszahlen
- Grafik
- Funktionen
- Benutzereingaben
- mathematische Berechnungen

Du hast damit einen wichtigen Meilenstein erreicht:

Nicht mehr einzelne Befehle stehen im Mittelpunkt, sondern ein vollständiges Programm.

Im nächsten Kapitel werden wir dieses Spiel weiter ausbauen und beispielsweise mehrere Schüsse, Highscores und Schwierigkeitsstufen hinzufügen.

Chapter 22. Treffe die Scheibe – Version 2

22.1. Das lernst du

In diesem Kapitel lernst du:

- mehrere Spielrunden durchzuführen
- Punkte zu vergeben
- Ergebnisse in Arrays zu speichern
- Statistiken auszuwerten
- Highscores zu speichern

Am Ende des Kapitels besitzt du ein kleines Grafikspiel mit Punktesystem.

22.2. Mehr Spielspaß

In der ersten Version durfte der Spieler nur einen einzigen Schuss abgeben.

Das ist zwar ein guter Anfang, aber noch kein richtiges Spiel.

In dieser Version erhält der Spieler zehn Schüsse.

Nach jedem Schuss werden Punkte vergeben.

Am Ende wird die Gesamtpunktzahl berechnet.

22.3. Schwierigkeitsstufen

Zu Beginn darf der Spieler den Schwierigkeitsgrad wählen.

```
PRINT "1 = Leicht"  
PRINT "2 = Mittel"  
PRINT "3 = Schwer"  
  
INPUT "Schwierigkeit ";LEVEL
```

Je höher die Schwierigkeit, desto kleiner wird die Zielscheibe.

```
IF LEVEL=1 THEN  
    RADIUS=60  
ELSEIF LEVEL=2 THEN  
    RADIUS=40  
ELSE  
    RADIUS=25  
ENDIF
```

22.4. Ergebnisse speichern

Für jeden Schuss speichern wir die Entfernung.

```
DIM TREFFER(9)
```

Damit können wir die Ergebnisse später auswerten.

22.5. Das komplette Spiel

```
MODE 2

CLS RGB(BLACK)

INPUT "Name ";NAME$

PRINT
PRINT "1 = Leicht"
PRINT "2 = Mittel"
PRINT "3 = Schwer"

INPUT "Schwierigkeit ";LEVEL

IF LEVEL=1 THEN
    RADIUS=60
ELSEIF LEVEL=2 THEN
    RADIUS=40
ELSE
    RADIUS=25
ENDIF

DIM TREFFER(9)

GESAMTPUNKTE=0

FOR RUNDE=1 TO 10

    CLS RGB(BLACK)

    TX=INT(RND*(MM.HRES-2*RADIUS))+RADIUS
    TY=INT(RND*(MM.VRES-2*RADIUS))+RADIUS

    ZeichneScheibe TX,TY,RADIUS

    PRINT
    PRINT "Schuss ";RUNDE;" von 10"

    INPUT "X ";SX
```



```

INPUT "Y ";SY

DIST=SQR((SX-TX)^2+(SY-TY)^2)

TREFFER(RUNDE-1)=DIST

CIRCLE SX,SY,4,1,1,RGB(RED),RGB(RED)

PUNKTE=100-INT(DIST)

IF PUNKTE<0 THEN
    PUNKTE=0
ENDIF

GESAMTPUNKTE=GESAMTPUNKTE+PUNKTE

PRINT
PRINT "Entfernung : ";INT(DIST);" Pixel"
PRINT "Punkte      : ";PUNKTE

PAUSE 1500

NEXT RUNDE

CLS RGB(BLACK)

PRINT
PRINT "===== "
PRINT "  SPIEL BEENDET"
PRINT "===== "
PRINT

PRINT NAME$
PRINT

PRINT "Gesamtpunkte : ";GESAMTPUNKTE

BEST=TREFFER(0)

FOR I=1 TO 9

    IF TREFFER(I)<BEST THEN
        BEST=TREFFER(I)
    ENDIF

NEXT I

PRINT
PRINT "Bester Schuss : ";INT(BEST);" Pixel"

PRINT

```

```

PRINT "Alle Schuesse"

FOR I=0 TO 9

    PRINT I+1;" . ";INT(TREFFER(I));" Pixel"

NEXT I

OPEN "highscore.txt" FOR OUTPUT AS #1

WRITE #1,NAME$,GESAMTPUNKTE

CLOSE #1

PRINT
PRINT "Highscore gespeichert"

END

SUB ZeichneScheibe(X,Y,R)

    CIRCLE X,Y,R,2,1,RGB(WHITE)

    CIRCLE X,Y,R*0.66,2,1,RGB(WHITE)

    CIRCLE X,Y,R*0.33,2,1,RGB(WHITE)

END SUB

```

22.6. Das Punktesystem

Die Entfernung wird in Punkte umgerechnet.

```
PUNKTE=100-INT(DIST)
```

Ein perfekter Treffer bringt 100 Punkte.

Je weiter der Schuss vom Mittelpunkt entfernt liegt, desto weniger Punkte gibt es.

22.7. Die Zielscheibe anpassen

Der Radius der Zielscheibe wird über die Schwierigkeit festgelegt.

Stufe	Radius
Leicht	60

Stufe	Radius
Mittel	40
Schwer	25

Dadurch wird das Spiel deutlich anspruchsvoller.

22.8. Die besten Schüsse ermitteln

Alle Entfernungen werden im Array **TREFFER()** gespeichert.

Am Ende wird der beste Schuss gesucht.

```
BEST=TREFFER(0)

FOR I=1 TO 9

    IF TREFFER(I)<BEST THEN
        BEST=TREFFER(I)
    ENDIF

NEXT I
```

Der kleinste Abstand entspricht dem besten Treffer.

22.9. Highscores speichern

Mit dem Wissen aus Kapitel 16 speichern wir den Highscore in einer Datei.

```
OPEN "highscore.txt" FOR OUTPUT AS #1

WRITE #1,NAME$,GESAMTPUNKTE

CLOSE #1
```

Die Datei enthält anschließend:

```
"Manfred",742
```

22.10. Beispielausgabe

```
=====
      SPIEL BEENDET
=====
```

Manfred

Gesamtpunkte : 742

Bester Schuss : 3 Pixel

Alle Schuesse

1. 12 Pixel
2. 45 Pixel
3. 3 Pixel
4. 27 Pixel
- ...

22.11. Experimentiere!

Probiere folgende Änderungen aus:

- Erhöhe die Anzahl der Schüsse auf 20.
- Verwende andere Farben.
- Vergib Bonuspunkte für Volltreffer.
- Speichere mehrere Highscores.

22.12. Probier's selbst!

Versuche folgende Erweiterungen:

1. Zeige die aktuelle Runde grafisch an.
2. Berechne die durchschnittliche Entfernung.
3. Speichere Datum und Uhrzeit mit dem Highscore.
4. Führe eine Bestenliste mit mehreren Spielern.

22.13. Zusammenfassung

In diesem Kapitel hast du das Spiel deutlich erweitert.

Du hast gelernt:

- Ergebnisse in Arrays zu speichern
- Punktesysteme zu entwickeln
- Statistiken auszuwerten
- Highscores zu speichern

Das Spiel fühlt sich nun deutlich mehr wie ein echtes Computerspiel an.

Im nächsten Kapitel werden wir die Steuerung weiter verbessern und die Zielscheibe direkt mit der Maus anvisieren.

Chapter 23. Treffe die Scheibe – Version 3

23.1. Das lernst du

In diesem Kapitel lernst du:

- die Maus in einem MMBasic-Programm abzufragen
- Mauskoordinaten für ein Grafikspiel zu verwenden
- einen Mausklick zu erkennen
- den Abstand zwischen Mausklick und Ziel zu berechnen
- das Spiel ohne die Eingabe von X- und Y-Koordinaten zu steuern

Am Ende des Kapitels kannst du die Zielscheibe direkt mit der Maus anvisieren.

23.2. Direkte Steuerung mit der Maus

In den bisherigen Versionen musste der Spieler die Koordinaten seines Schusses über die Tastatur eingeben:

```
X ? 120  
Y ? 85
```

Das war nützlich, um das Koordinatensystem kennenzulernen.

Für ein Grafikspiel ist es jedoch viel natürlicher, direkt mit der Maus auf den Bildschirm zu klicken.

In Version 3 ersetzen wir deshalb die Koordinateneingabe durch eine Maussteuerung.

23.3. Informationen der Maus abfragen

MMBasic stellt über die Funktion `MOUSE()` verschiedene Informationen über die angeschlossene Maus bereit.

Die aktuelle horizontale Position erhältst du mit:

```
MOUSE(X)
```

Die aktuelle vertikale Position erhältst du mit:

```
MOUSE(Y)
```

Der Zustand der linken Maustaste kann mit

MOUSE(L)

abgefragt werden.

Ein Wert ungleich 0 bedeutet, dass die linke Maustaste gedrückt ist.



Die Maus muss vom verwendeten PicoMite-System unterstützt und korrekt angeschlossen beziehungsweise konfiguriert sein.

Das folgende Programm ist insbesondere für PicoMite-Systeme mit VGA- oder HDMI-Ausgabe und Mausunterstützung vorgesehen.

23.4. Die aktuelle Mausposition anzeigen

Mit einem kleinen Testprogramm kannst du zunächst die Mauskoordinaten beobachten.

```
DO  
  
  LOCATE 1,1  
  
  PRINT "X: ";MOUSE(X);"   "  
  PRINT "Y: ";MOUSE(Y);"   "  
  
LOOP
```

Bewege die Maus über den Bildschirm.

Die angezeigten Werte verändern sich entsprechend der aktuellen Position.

Das Programm kann mit **Strg+C** beendet werden.

23.5. Auf einen Mausklick warten

Das folgende Beispiel wartet, bis die linke Maustaste gedrückt wird:

```
DO  
LOOP UNTIL MOUSE(L)<>0  
  
PRINT "Maus wurde geklickt."
```

Die Schleife wird so lange wiederholt, bis ein Mausklick erkannt wird.

23.6. Die Position eines Mausklicks speichern

Beim Mausklick speichern wir die aktuellen Koordinaten:

```
DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
SY=MOUSE(Y)
```

Die Variablen **SX** und **SY** enthalten anschließend die Position des Schusses.

23.7. Auf das Loslassen der Maustaste warten

Eine Maustaste bleibt meist für mehrere Programmdurchläufe gedrückt.

Damit ein einzelner Klick nicht mehrfach erkannt wird, warten wir anschließend auf das Loslassen:

```
DO
LOOP UNTIL MOUSE(L)=0
```

Ein vollständiger Mausklick besteht damit aus zwei Schritten:

```
DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
SY=MOUSE(Y)

DO
LOOP UNTIL MOUSE(L)=0
```

23.8. Der neue Spielablauf

Das Spiel besteht wieder aus zehn Runden.

In jeder Runde geschieht Folgendes:

1. Der Bildschirm wird gelöscht.
2. Eine Zielscheibe wird an einer zufälligen Position gezeichnet.
3. Das Programm wartet auf einen Mausklick.
4. Der Klick wird als Trefferpunkt dargestellt.
5. Entfernung und Punktzahl werden berechnet.
6. Das Ergebnis wird gespeichert.

23.9. Das komplette Spiel

```
MODE 2

INPUT "Name ";NAME$

PRINT
PRINT "1 = Leicht"
PRINT "2 = Mittel"
PRINT "3 = Schwer"

INPUT "Schwierigkeit ";LEVEL

IF LEVEL=1 THEN
    RADIUS=60
ELSEIF LEVEL=2 THEN
    RADIUS=40
ELSE
    RADIUS=25
ENDIF

DIM TREFFER(9)
DIM PUNKTE(9)

GESAMTPUNKTE=0

FOR RUNDE=1 TO 10

    CLS RGB(BLACK)

    TX=INT(RND*(MM.HRES-2*RADIUS))+RADIUS
    TY=INT(RND*(MM.VRES-2*RADIUS))+RADIUS

    ZeichneScheibe TX,TY,RADIUS

    TEXT MM.HRES/2,15,"Schuss "+STR$(RUNDE)+" von 10","CT",1,1,RGB(WHITE),-1

    TEXT MM.HRES/2,MM.VRES-20,"Klicke auf die Zielscheibe","CB",1,1,RGB(YELLOW),-1

    DO
        LOOP UNTIL MOUSE(L)<>0

    SX=MOUSE(X)
    SY=MOUSE(Y)

    DO
        LOOP UNTIL MOUSE(L)=0

    DIST=SQR((SX-TX)^2+(SY-TY)^2)
```

```

TREFFER(RUNDE-1)=DIST

PUNKTE(RUNDE-1)=100-INT(DIST)

IF PUNKTE(RUNDE-1)<0 THEN
    PUNKTE(RUNDE-1)=0
ENDIF

GESAMTPUNKTE=GESAMTPUNKTE+PUNKTE(RUNDE-1)

ZeichneTreffer SX,SY

TEXT MM.HRES/2,MM.VRES-55,
    STR$(INT(DIST))+ " Pixel entfernt","CB",
    1,1,RGB(WHITE),-1

TEXT MM.HRES/2,MM.VRES-35,
    STR$(PUNKTE(RUNDE-1))+ " Punkte","CB",
    1,1,RGB(CYAN),-1

IF DIST<=RADIUS*0.33 THEN

    TEXT MM.HRES/2,MM.VRES/2,
        "VOLLTREFFER!","CM",2,2,RGB(YELLOW),-1

ELSEIF DIST<=RADIUS*0.66 THEN

    TEXT MM.HRES/2,MM.VRES/2,
        "SEHR GUT!","CM",2,2,RGB(GREEN),-1

ELSEIF DIST<=RADIUS THEN

    TEXT MM.HRES/2,MM.VRES/2,
        "GETROFFEN!","CM",2,2,RGB(CYAN),-1

ELSE

    TEXT MM.HRES/2,MM.VRES/2,
        "DANEHEN!","CM",2,2,RGB(RED),-1

ENDIF

PAUSE 1500

NEXT RUNDE

ZeigeAuswertung

SpeichereErgebnis

END

```

```
SUB ZeichneScheibe(X,Y,R)
```

```
    CIRCLE X,Y,R,2,1,RGB(WHITE),RGB(BLUE)
```

```
    CIRCLE X,Y,R*0.66,2,1,RGB(WHITE),RGB(RED)
```

```
    CIRCLE X,Y,R*0.33,2,1,RGB(WHITE),RGB(YELLOW)
```

```
    CIRCLE X,Y,3,1,1,RGB(BLACK),RGB(BLACK)
```

```
END SUB
```

```
SUB ZeichneTreffer(X,Y)
```

```
    LINE X-7,Y,X+7,Y,2,RGB(WHITE)
```

```
    LINE X,Y-7,X,Y+7,2,RGB(WHITE)
```

```
    CIRCLE X,Y,4,1,1,RGB(BLACK),RGB(RED)
```

```
END SUB
```

```
SUB ZeigeAuswertung
```

```
    CLS RGB(BLACK)
```

```
    TEXT MM.HRES/2,20,"SPIEL BEENDET","CT",2,2,RGB(YELLOW),-1
```

```
    TEXT MM.HRES/2,65,NAME$,"CT",1,2,RGB(WHITE),-1
```

```
    TEXT MM.HRES/2,100,                                -  
        "Gesamtpunkte: "+STR$(GESAMTPUNKTE),          -  
        "CT",1,2,RGB(CYAN),-1
```

```
    BEST=TREFFER(0)
```

```
    SUMME=0
```

```
    FOR I=0 TO 9
```

```
        SUMME=SUMME+TREFFER(I)
```

```
        IF TREFFER(I)<BEST THEN
```

```
            BEST=TREFFER(I)
```

```
        ENDIF
```

```

NEXT I

DURCHSCHNITT=SUMME/10

TEXT MM.HRES/2,135,
    "Bester Schuss: "+STR$(INT(BEST))+ " Pixel", _
    "CT",1,1,RGB(GREEN),-1

TEXT MM.HRES/2,160,
    "Durchschnitt: "+
    STR$(INT(DURCHSCHNITT))+ " Pixel",
    "CT",1,1,RGB(WHITE),-1

Y=200

FOR I=0 TO 9

    TEXT MM.HRES/2,Y,
        STR$(I+1)+". Schuss: "+
        STR$(INT(TREFFER(I)))+ " Pixel, "+
        STR$(PUNKTE(I))+ " Punkte",
        "CT",1,1,RGB(WHITE),-1

    Y=Y+18

NEXT I

END SUB

SUB SpeichereErgebnis

    OPEN "highscore.txt" FOR APPEND AS #1

    WRITE #1,DATE$,TIME$,NAME$,LEVEL,GESAMTPUNKTE

    CLOSE #1

END SUB

```

23.10. Das Programm verstehen

Die bisherige Eingabe

```

INPUT "X ";SX
INPUT "Y ";SY

```

wurde durch die Mausabfrage ersetzt:

```
DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
SY=MOUSE(Y)

DO
LOOP UNTIL MOUSE(L)=0
```

Dadurch wird der Schuss genau an der angeklickten Bildschirmposition abgegeben.

23.11. Den Trefferpunkt darstellen

Ein kleiner roter Kreis zeigt die genaue Position des Mausklicks.

Zusätzlich zeichnen zwei Linien ein Fadenkreuz:

```
LINE X-7,Y,X+7,Y,2,RGB(WHITE)

LINE X,Y-7,X,Y+7,2,RGB(WHITE)

CIRCLE X,Y,4,1,1,RGB(BLACK),RGB(RED)
```

Dadurch ist der Treffer auch auf einer farbigen Zielscheibe gut sichtbar.

23.12. Trefferbereiche auswerten

Die Größe der Zielscheibe hängt von der gewählten Schwierigkeitsstufe ab.

Deshalb werden die Trefferbereiche relativ zum Radius berechnet.

```
IF DIST<=RADIUS*0.33 THEN
...
ELSEIF DIST<=RADIUS*0.66 THEN
...
ELSEIF DIST<=RADIUS THEN
...
ELSE
...
ENDIF
```

So funktioniert die Auswertung bei allen drei Schwierigkeitsstufen.

23.13. Punkte und Entfernungen speichern

Für jede Runde werden Entfernung und Punktzahl in Arrays gespeichert:

```
TREFFER(RUNDE-1)=DIST  
PUNKTE(RUNDE-1)=100-INT(DIST)
```

Nach dem Spiel können diese Werte ausgewertet und angezeigt werden.

23.14. Den Durchschnitt berechnen

Die Summe aller Entfernungen wird durch die Anzahl der Schüsse geteilt:

```
SUMME=0  
  
FOR I=0 TO 9  
    SUMME=SUMME+TREFFER(I)  
NEXT I  
  
DURCHSCHNITT=SUMME/10
```

Je kleiner der Durchschnitt ist, desto genauer hat der Spieler insgesamt gezielt.

23.15. Ergebnisse dauerhaft speichern

Version 2 hatte die Highscore-Datei bei jedem Spiel überschrieben.

Version 3 verwendet stattdessen den Modus **APPEND**:

```
OPEN "highscore.txt" FOR APPEND AS #1
```

Dadurch bleibt jedes Spielergebnis erhalten.

Mit **WRITE** werden gespeichert:

- Datum
- Uhrzeit
- Spielername
- Schwierigkeitsstufe
- Gesamtpunktzahl

Ein Eintrag kann beispielsweise so aussehen:

23.16. Warum SUB hier besonders hilfreich ist

Das Programm besteht inzwischen aus mehreren klar getrennten Aufgaben:

```
ZeichneScheibe  
ZeichneTreffer  
ZeigeAuswertung  
SpeichereErgebnis
```

Jede dieser Aufgaben wurde in eine eigene **SUB** ausgelagert.

Dadurch bleibt das Hauptprogramm trotz seines größeren Umfangs übersichtlich.

23.17. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende fünf statt zehn Schüsse.
- Ändere die Farben der Zielscheibe.
- Verkleinere den Radius der schweren Stufe.
- Vergib 200 statt 100 maximal mögliche Punkte.
- Zeichne einen eigenen Mauszeiger.
- Zeige während des Spiels die Gesamtpunktzahl an.

23.18. Probier's selbst!

Versuche folgende Erweiterungen:

1. Beende das Spiel mit der rechten Maustaste.
2. Zeichne vor jedem Schuss einen Countdown.
3. Begrenze die Zeit für einen Schuss.
4. Vergib Bonuspunkte für schnelle Treffer.
5. Lade den bisherigen Highscore aus der Datei.
6. Zeige eine Bestenliste mit mehreren Spielern an.
7. Lasse die Zielscheibe nach kurzer Zeit ihre Position ändern.

23.19. Zusammenfassung

In diesem Kapitel hast du das Grafikspiel auf Maussteuerung umgestellt.

Du hast gelernt:

- Mauskoordinaten abzufragen
- Mausklicks zu erkennen
- Klickpositionen grafisch darzustellen
- Treffer relativ zur Zielgröße auszuwerten
- Ergebnisse in Arrays zu speichern
- Statistiken zu berechnen
- mehrere Highscores dauerhaft zu sichern

Aus einem einfachen Koordinatenexperiment ist damit Schritt für Schritt ein vollständiges kleines Grafikspiel entstanden.

Chapter 24. Eine analoge Bahnhofsuhr

24.1. Das lernst du

In diesem Kapitel lernst du:

- eine bildschirmfüllende analoge Uhr zu zeichnen
- die aktuelle Uhrzeit aus `TIME$` auszulesen
- Winkel in Bildschirmkoordinaten umzurechnen
- Stunden-, Minuten- und Sekundenzeiger darzustellen
- ein Zifferblatt an unterschiedliche Bildschirmauflösungen anzupassen
- mehrere Grafikbefehle zu einer dynamischen Anzeige zu verbinden

Am Ende des Kapitels besitzt du eine laufende analoge Uhr im Stil einer klassischen Bahnhofsuhr.

24.2. Die Idee

Klassische Bahnhofsuhren besitzen ein besonders klares und gut lesbares Design:

- ein weißes Zifferblatt
- einen schwarzen Rand
- kräftige Stundenmarkierungen
- kleinere Minutenmarkierungen
- schwarze Stunden- und Minutenzeiger
- einen auffälligen roten Sekundenzeiger

Auf Ziffern wird häufig verzichtet.

Dadurch lässt sich die Uhrzeit auch aus größerer Entfernung schnell erkennen.

Unsere MMBasic-Uhr soll sich automatisch an die aktuelle Bildschirmgröße anpassen und möglichst viel von der verfügbaren Fläche nutzen.

24.3. Die Bildschirmmitte bestimmen

Zuerst bestimmen wir den Mittelpunkt des Bildschirms.

```
MITTEX=MM.HRES/2  
MITTEY=MM.VRES/2
```

Die Variablen `MM.HRES` und `MM.VRES` enthalten die aktuelle Bildschirmbreite und Bildschirmhöhe.

24.4. Den größtmöglichen Radius berechnen

Die Uhr soll weder rechts noch unten aus dem Bildschirm herausragen.

Deshalb berechnen wir zunächst den Radius aus der Bildschirmbreite:

```
RADIUS=MM.HRES/2-10
```

Danach prüfen wir, ob die Bildschirmhöhe einen kleineren Radius erfordert:

```
IF MM.VRES/2-10<RADIUS THEN  
  RADIUS=MM.VRES/2-10  
ENDIF
```

Damit bleibt rund um die Uhr ein kleiner Rand von zehn Pixeln erhalten.

24.5. Das Zifferblatt zeichnen

Das Zifferblatt besteht aus einem großen weißen Kreis mit einem schwarzen Rand.

```
CIRCLE MITTEX,MITTEY,RADIUS,3,1,RGB(BLACK),RGB(WHITE)
```

Der Kreis besitzt:

- den Mittelpunkt **MITTEX**, **MITTEY**
- den berechneten Radius
- einen schwarzen Rand
- eine weiße Füllung

24.6. Die Minutenmarkierungen

Eine Uhr besitzt 60 Markierungen.

Der Abstand zwischen zwei Markierungen beträgt:

```
360 Grad / 60 = 6 Grad
```

Mit einer Schleife können wir alle Markierungen erzeugen.

```
FOR I=0 TO 59  
  
  WINKEL=I*6*PI/180
```

Die Funktionen `SIN()` und `COS()` erwarten Winkel im Bogenmaß.

Deshalb wird der Winkel mit `PI/180` umgerechnet.

24.7. Positionen auf einem Kreis berechnen

Für jede Markierung benötigen wir zwei Punkte:

- den Anfang der Markierung
- das Ende der Markierung

Die äußere Position berechnen wir so:

```
X2=MITTEX+SIN(WINKEL)*RADIUS*0.92
Y2=MITTEY-COS(WINKEL)*RADIUS*0.92
```

Das Minuszeichen vor `COS()` ist notwendig, weil die Y-Koordinate auf dem Bildschirm nach unten wächst.

24.8. Stunden- und Minutenmarkierungen unterscheiden

Jede fünfte Markierung steht für eine volle Stunde.

Diese Markierungen werden länger gezeichnet.

```
IF I MOD 5=0 THEN
  INNEN=RADIUS*0.78
ELSE
  INNEN=RADIUS*0.86
ENDIF
```

Anschließend werden Anfang und Ende berechnet:

```
X1=MITTEX+SIN(WINKEL)*INNEN
Y1=MITTEY-COS(WINKEL)*INNEN

X2=MITTEX+SIN(WINKEL)*RADIUS*0.92
Y2=MITTEY-COS(WINKEL)*RADIUS*0.92

LINE X1,Y1,X2,Y2,,,RGB(BLACK)
```

24.9. Die Uhrzeit aus TIME\$ auslesen

TIME\$ liefert die aktuelle Uhrzeit als Zeichenkette.

Ein typischer Wert ist:

21:37:45

Die Stunde steht in den ersten beiden Zeichen:

STUNDE=VAL(LEFT\$(TIME\$,2))

Die Minute beginnt an Position 4:

MINUTE=VAL(MID\$(TIME\$,4,2))

Die Sekunde steht am Ende:

SEKUNDE=VAL(RIGHT\$(TIME\$,2))

Mit VAL() werden die Zeichenketten in Zahlen umgewandelt.

24.10. Den Winkel des Sekundenzeigers berechnen

Eine vollständige Umdrehung besitzt 360 Grad.

Der Sekundenzeiger bewegt sich in 60 Schritten:

360 Grad / 60 Sekunden = 6 Grad pro Sekunde

Der Winkel lautet deshalb:

SEKUNDENWINKEL=SEKUNDE*6*PI/180

24.11. Den Winkel des Minutenzeigers berechnen

Der Minutenzeiger bewegt sich ebenfalls um sechs Grad pro Minute.

Zusätzlich berücksichtigen wir die Sekunden, damit sich der Zeiger gleichmäßiger bewegt.

MINUTENWINKEL=(MINUTE*6+SEKUNDE*0.1)*PI/180

24.12. Den Winkel des Stundenzeigers berechnen

Der Stundenzeiger bewegt sich pro Stunde um 30 Grad:

$$360 \text{ Grad} / 12 \text{ Stunden} = 30 \text{ Grad pro Stunde}$$

Er soll sich auch zwischen zwei Stundenmarkierungen weiterbewegen.

$$\text{STUNDENWINKEL} = ((\text{STUNDE} \bmod 12) * 30 + \text{MINUTE} * 0.5) * \pi / 180$$

Durch **MOD 12** werden beispielsweise 13 Uhr und 1 Uhr auf dieselbe Position abgebildet.

24.13. Einen Zeiger zeichnen

Für einen Zeiger benötigen wir dessen Endpunkt.

Der Endpunkt des Stundenzeigers wird so berechnet:

$$\begin{aligned} X &= \text{MITTEX} + \sin(\text{STUNDENWINKEL}) * \text{RADIUS} * 0.50 \\ Y &= \text{MITTEY} - \cos(\text{STUNDENWINKEL}) * \text{RADIUS} * 0.50 \end{aligned}$$

Anschließend wird die Linie vom Mittelpunkt zum Endpunkt gezeichnet:

$$\text{LINE MITTEX, MITTEY, X, Y, , , RGB(BLACK)}$$

Der Minutenzeiger ist länger:

$$\begin{aligned} X &= \text{MITTEX} + \sin(\text{MINUTENWINKEL}) * \text{RADIUS} * 0.72 \\ Y &= \text{MITTEY} - \cos(\text{MINUTENWINKEL}) * \text{RADIUS} * 0.72 \end{aligned}$$

Der Sekundenzeiger reicht fast bis zu den Minutenmarkierungen:

$$\begin{aligned} X &= \text{MITTEX} + \sin(\text{SEKUNDENWINKEL}) * \text{RADIUS} * 0.84 \\ Y &= \text{MITTEY} - \cos(\text{SEKUNDENWINKEL}) * \text{RADIUS} * 0.84 \end{aligned}$$

24.14. Der rote Sekundenzeiger

Der rote Sekundenzeiger ist ein besonders auffälliges Merkmal klassischer Bahnhofsuhren.

$$\text{LINE MITTEX, MITTEY, X, Y, , , RGB(RED)}$$

Zusätzlich zeichnen wir kurz vor der Spitze einen roten Kreis.

```
KX=MITTEX+SIN(SEKUNDENWINKEL)*RADIUS*0.70
KY=MITTEY-COS(SEKUNDENWINKEL)*RADIUS*0.70

CIRCLE KX,KY,RADIUS*0.035,1,1,RGB(RED),RGB(RED)
```

Dadurch erhält der Sekundenzeiger seine typische Form.

24.15. Der Mittelpunkt

Zum Abschluss wird der Drehpunkt der Zeiger hervorgehoben.

```
CIRCLE MITTEX,MITTEY,RADIUS*0.035,1,1,RGB(BLACK),RGB(BLACK)
```

24.16. Das vollständige Programm

```
MITTEX=MM.HRES/2
MITTEY=MM.VRES/2

RADIUS=MM.HRES/2-10

IF MM.VRES/2-10<RADIUS THEN
    RADIUS=MM.VRES/2-10
ENDIF

ALTEZEIT$=""

DO

    IF TIME$<>ALTEZEIT$ THEN

        ALTEZEIT$=TIME$

        STUNDE=VAL(LEFT$(TIME$,2))
        MINUTE=VAL(MID$(TIME$,4,2))
        SEKUNDE=VAL(RIGHT$(TIME$,2))

        CLS RGB(WHITE)

        ZeichneZifferblatt MITTEX,MITTEY,RADIUS

        STUNDENWINKEL=((STUNDE MOD 12)*30+MINUTE*0.5)*PI/180

        MINUTENWINKEL=(MINUTE*6+SEKUNDE*0.1)*PI/180

        SEKUNDENWINKEL=SEKUNDE*6*PI/180
```

```

ZeichneStundenzeiger MITTEX,MITTEY,RADIUS,STUNDENWINKEL

ZeichneMinutenzeiger MITTEX,MITTEY,RADIUS,MINUTENWINKEL

ZeichneSekundenzeiger MITTEX,MITTEY,RADIUS,SEKUNDENWINKEL

CIRCLE MITTEX,MITTEY,RADIUS*0.035,1,1,RGB(BLACK),RGB(BLACK)

ENDIF

LOOP

SUB ZeichneZifferblatt(X,Y,R)

CIRCLE X,Y,R,3,1,RGB(BLACK),RGB(WHITE)

FOR I=0 TO 59

    WINKEL=I*6*PI/180

    IF I MOD 5=0 THEN
        INNEN=R*0.78
    ELSE
        INNEN=R*0.86
    ENDIF

    X1=X+SIN(WINKEL)*INNEN
    Y1=Y-COS(WINKEL)*INNEN

    X2=X+SIN(WINKEL)*R*0.92
    Y2=Y-COS(WINKEL)*R*0.92

    LINE X1,Y1,X2,Y2,,,RGB(BLACK)

NEXT I

END SUB

SUB ZeichneStundenzeiger(X,Y,R,WINKEL)

X2=X+SIN(WINKEL)*R*0.50
Y2=Y-COS(WINKEL)*R*0.50

LINE X,Y,X2,Y2,,,RGB(BLACK)

LINE X-1,Y,X2-1,Y2,,,RGB(BLACK)

```

```

LINE X+1,Y,X2+1,Y2,,,RGB(BLACK)

END SUB

SUB ZeichneMinutenzeiger(X,Y,R,WINKEL)

X2=X+SIN(WINKEL)*R*0.72
Y2=Y-COS(WINKEL)*R*0.72

LINE X,Y,X2,Y2,,,RGB(BLACK)

LINE X-1,Y,X2-1,Y2,,,RGB(BLACK)
LINE X+1,Y,X2+1,Y2,,,RGB(BLACK)

END SUB

SUB ZeichneSekundenzeiger(X,Y,R,WINKEL)

X2=X+SIN(WINKEL)*R*0.84
Y2=Y-COS(WINKEL)*R*0.84

LINE X,Y,X2,Y2,,,RGB(RED)

KX=X+SIN(WINKEL)*R*0.70
KY=Y-COS(WINKEL)*R*0.70

CIRCLE KX,KY,R*0.035,1,1,RGB(RED),RGB(RED)

END SUB

```

24.17. Das Programm verstehen

Das Hauptprogramm wartet in einer Endlosschleife auf eine Änderung von **TIME\$**.

```
IF TIME$<>ALTEZEIT$ THEN
```

Die Uhr wird dadurch nur einmal pro Sekunde neu gezeichnet.

Anschließend werden:

1. das Zifferblatt
2. der Stundenzeiger
3. der Minutenzeiger

4. der Sekundenzeiger
5. der Mittelpunkt

nacheinander dargestellt.

24.18. Warum wird der Bildschirm neu gezeichnet?

Die alten Zeiger müssen entfernt werden, bevor die neue Zeigerposition dargestellt wird.

Die einfachste Lösung besteht darin, den Bildschirm einmal pro Sekunde zu löschen und die vollständige Uhr neu zu zeichnen.

```
CLS RGB(WHITE)
```

Bei schnellen Systemen geschieht dies nahezu augenblicklich.



Je nach Bildschirm, Grafikmodus und Prozessorgeschwindigkeit kann beim Neuzeichnen ein leichtes Flackern sichtbar sein.

Später lernen wir Framebuffer kennen. Damit kann eine vollständige Grafik zunächst unsichtbar aufgebaut und anschließend in einem Schritt auf den Bildschirm übertragen werden.

24.19. Auflösungsunabhängige Darstellung

Das Programm verwendet keine fest vorgegebenen Mittelpunkt- oder Radiuswerte.

Stattdessen werden sie aus

```
MM.HRES  
MM.VRES
```

berechnet.

Dadurch passt sich die Uhr automatisch an unterschiedliche Bildschirmauflösungen an.

24.20. Eine digitale Ergänzung

Unterhalb der Uhr kann zusätzlich die digitale Uhrzeit ausgegeben werden.

Füge nach dem Zeichnen des Mittelpunkts ein:

```
TEXT MITTEX,MITTEY+RADIUS*0.55,TIME$,"CM",1,1,RGB(BLACK),-1
```

Dadurch werden analoge und digitale Anzeige kombiniert.

24.21. Einen Schriftzug ergänzen

Klassische Bahnhofsuhren besitzen häufig einen kleinen Schriftzug.

```
TEXT MITTEX,MITTEY-RADIUS*0.45,"MMBasic","CM",1,1,RGB(BLACK),-1
```

Du kannst dort beispielsweise deinen Namen oder die Bezeichnung deines PicoMite eintragen.

24.22. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende einen blauen Sekundenzeiger.
- Vergrößere den roten Kreis am Sekundenzeiger.
- Ergänze die Ziffern 3, 6, 9 und 12.
- Zeige zusätzlich das aktuelle Datum an.
- Ändere die Länge der Zeiger.
- Verwende einen schwarzen Hintergrund und ein weißes Zifferblatt.

24.23. Probier's selbst!

Versuche folgende Erweiterungen:

1. Zeichne die Stundenmarkierungen kräftiger.
2. Ergänze einen äußeren Schatten um das Zifferblatt.
3. Gib unterhalb der Uhr `DATE$` aus.
4. Erstelle eine Nachtansicht mit dunklem Zifferblatt.
5. Ergänze eine zweite Zeitzone.
6. Spiele zu jeder vollen Stunde einen Ton ab.
7. Verwende später einen Framebuffer, um das Flackern zu vermeiden.

24.24. Zusammenfassung

In diesem Kapitel hast du eine vollständige analoge Uhr programmiert.

Du hast gelernt:

- eine Grafik an die Bildschirmauflösung anzupassen
- kreisförmig angeordnete Markierungen zu berechnen
- `TIME$` in Stunden, Minuten und Sekunden zu zerlegen
- Winkel mit `SIN()` und `COS()` in Koordinaten umzuwandeln

- bewegliche Zeiger darzustellen
- ein dynamisches Grafikprogramm zu strukturieren

Die Bahnhofsuhr verbindet Grafik, Mathematik, Zeichenketten, Schleifen und Unterprogramme zu einer vollständigen Anwendung.